



**Kafr El-Sheikh University**

**Faculty of Computers and Information**

Computer Science Department

**Graduation Project Book**

2025 / 2026

# **CareerCompass: AI-Powered Career Guidance and Job Recommendation Platform**

**CareerCompass**

**Submitted by:**

Yousef Altohamy Ahmed Altohamy

Ahmed Mohamed Ahmed Abdelaziz

Mohamed Ali Ahmed Mohamed

Mohamed Ibrahim Ahmed Mohamed

Ahmed Khamis Mohamed Younes

Ahmed Sobhy Mohamed Ali

**Supervised by:**

Dr. Amna Mahmoud

# Table of Contents

- [List of Figures](#)
- [List of Tables](#)
- [Acknowledgment](#)
- [Abstract](#)
- [Abbreviations](#)
- [Chapter 1: Introduction](#)
- [Chapter 2: System Analysis](#)
- [Chapter 3: System Design and Architecture](#)
- [Chapter 4: Software and Tools Used](#)
- [Chapter 5: System Implementation](#)
- [Chapter 6: Testing and Evaluation](#)
- [Chapter 7: Security and Privacy](#)
- [Chapter 8: Conclusion and Future Work](#)
- [References](#)
- [Appendices](#)

## List of Figures

- [Figure 1. High-level architecture of CareerCompass.](#)
- [Figure 2. Docker deployment architecture.](#)
- [Figure 3. DFD Level 0 context diagram.](#)
- [Figure 4. DFD Level 1 process diagram.](#)
- [Figure 5. UML use case diagram.](#)
- [Figure 6. Sequence diagram for CV upload and analysis.](#)
- [Figure 7. Sequence diagram for recommendation and gap analysis.](#)
- [Figure 8. ERD and database summary diagram.](#)
- [Figure 9. Home page.](#)
- [Figure 10. Register page.](#)
- [Figure 11. Login page.](#)
- [Figure 12. Student dashboard before CV upload.](#)
- [Figure 13. CV upload user interface.](#)
- [Figure 14. Dashboard after successful CV parsing.](#)
- [Figure 15. Extracted profile and skills page.](#)
- [Figure 16. Jobs recommendations page.](#)
- [Figure 17. Job detail and inline gap panel.](#)
- [Figure 18. Gap analysis page.](#)
- [Figure 19. Applications tracker page.](#)
- [Figure 20. Tools Hub preview page.](#)
- [Figure 21. System status page.](#)
- [Figure 22. Admin dashboard.](#)
- [Figure 23. Admin jobs page.](#)
- [Figure 24. Admin sources diagnostics page.](#)
- [Figure 25. Admin target roles page.](#)
- [Figure 26. Docker services evidence.](#)
- [Figure 27. Validation command evidence.](#)

## List of Tables

- [Table 1. Stakeholder summary.](#)
- [Table 2. Functional requirements summary.](#)
- [Table 3. Non-functional requirements summary.](#)
- [Table 4. Hardware and software environment.](#)
- [Table 5. Design decisions summary.](#)
- [Table 6. Mini CV dataset.](#)
- [Table 7. Mini job dataset.](#)
- [Table 8. Mini evaluation metrics.](#)
- [Table 9. Recommendation ranking details.](#)
- [Table 10. Gap analysis pair details.](#)
- [Table 11. Automated validation results.](#)
- [Table 12. Manual functional evaluation matrix.](#)
- [Table 13. Manual functional observations.](#)
- [Table 14. Security and privacy controls.](#)
- [Table 15. API endpoint summary.](#)
- [Table 16. Database tables summary.](#)
- [Table 17. Docker services summary.](#)

## Acknowledgment

The project team would like to express sincere appreciation to Dr. Amna Mahmoud for academic supervision, technical guidance, and continuous feedback during the preparation of CareerCompass. The team also thanks the Faculty of Computers and Information at Kafr El-Sheikh University for providing the academic setting in which this graduation project was designed, implemented, tested, and documented.

The work presented in this book reflects a collaborative software engineering effort. It combines web application development, database design, AI-assisted document analysis, explainable matching, containerized deployment, testing, and technical documentation. The two supervisor-provided graduation books were used only to understand expected report structure and visual formality; no content, wording, project-specific claims, diagrams, or references were copied from them.

## Abstract

CareerCompass is a graduation/demo career guidance platform that helps students and early-career users understand their CV profile, explore imported job opportunities, and compare their current skills against job requirements. The system consists of a React and Vite frontend, a Laravel API backend, a MySQL database, a FastAPI-based CV analyzer, a FastAPI/Scrapy-based job miner, MinIO-compatible private file storage, Nginx routing, and Prometheus/Grafana monitoring. The platform supports registration, login, CV upload, AI-assisted CV parsing, normalized profile and skills storage, job recommendation, gap analysis, an application tracker, and administrator dashboards for job and source diagnostics.

The implementation is intentionally described as a graduation/demo system rather than a production product. The AI outputs are estimates, the job data depends on imported and demo sources, and the security posture is appropriate for demonstration but requires further production hardening. Validation was performed through Docker Compose configuration checks, backend tests, frontend lint/build, Python service tests or syntax checks, HTTP probes, and manual browser screenshots. Backend tests passed with 39 tests and 297 assertions, the AI job miner tests passed with 75 tests, and the frontend build completed successfully. The AI CV analyzer container did not include pytest, so its pytest suite was marked as skipped while Python syntax compilation passed.

## Abbreviations

| Abbreviation | Meaning                                          |
|--------------|--------------------------------------------------|
| API          | Application Programming Interface                |
| CV           | Curriculum Vitae                                 |
| DFD          | Data Flow Diagram                                |
| ERD          | Entity Relationship Diagram                      |
| HTTP         | Hypertext Transfer Protocol                      |
| JSON         | JavaScript Object Notation                       |
| JWT          | JSON Web Token                                   |
| ML           | Machine Learning                                 |
| NLP          | Natural Language Processing                      |
| OCR          | Optical Character Recognition                    |
| RBAC         | Role-Based Access Control                        |
| REST         | Representational State Transfer                  |
| S3           | Simple Storage Service compatible object storage |
| TF-IDF       | Term Frequency-Inverse Document Frequency        |
| UI           | User Interface                                   |

# Chapter 1: Introduction

## 1.1 Introduction to the Project

CareerCompass is an AI-assisted career guidance and job recommendation platform developed as a Computer Science graduation project for Kafr El-Sheikh University. The system helps a student upload a CV, receive a structured profile, view estimated job matches, inspect skill gaps, and save opportunities in an application tracker. The project combines web engineering, backend service design, natural language processing support, web scraping support, data persistence, and containerized operations.

The platform uses a modern web stack. The frontend is implemented with React, a component-based UI library [4], and bundled with Vite [5]. The backend uses Laravel, which provides routing, controllers, validation, middleware, Eloquent ORM, queues, testing support, and file storage abstractions [1]. The AI services are implemented with FastAPI, a Python API framework that supports type-hinted request and response handling [6]. The deployment is orchestrated through Docker Compose [11].

## 1.2 Background and Motivation

Many students prepare CVs and search for jobs without a clear view of how their current skills relate to job descriptions. Career guidance is often fragmented across CV feedback, job boards, learning resources, and manual advice. CareerCompass addresses this academic problem by placing those steps into one integrated demonstration system. The objective is not to replace human career advising, but to provide a practical software prototype that can parse a CV, normalize profile data, import job records, estimate matches, and present explainable gap analysis.

## 1.3 Problem Statement

Students frequently face three connected problems. First, they may not know whether their CV communicates their skills clearly. Second, job listings often describe requirements in different formats, making comparison difficult. Third, students may save opportunities across separate tools without a coherent tracker. CareerCompass proposes a centralized graduation/demo system that links CV data, jobs, matching, gap analysis, and tracking.

## 1.4 Purpose of the Project

The purpose of CareerCompass is to demonstrate how a distributed web system can support career exploration using explainable AI-assisted workflows. The project demonstrates authentication, CV upload, private storage, parsing, skill extraction, job import, recommendation, gap analysis, admin diagnostics, and monitoring in a Dockerized environment.

## 1.5 Project Objectives

- Provide a student-facing web interface for account creation, login, CV upload, profile review, job recommendations, gap analysis, and application tracking.
- Provide an administrator interface for dashboard statistics, job review, source diagnostics, and target role management.
- Implement a Laravel API that protects authenticated workflows using token-based access and role checks.
- Implement Python services for CV analysis and job mining support.
- Store uploaded CV files privately and expose downloads through controlled URLs.
- Validate the system with backend tests, frontend lint/build, Python tests where available, Docker checks, HTTP probes, and browser screenshots.

## 1.6 Proposed Solution

The proposed solution is a multi-service application. React renders the browser interface. Nginx serves as the gateway. Laravel handles REST-style APIs over HTTP [16], authentication, validation, data models, business services, and admin routes. MySQL stores normalized data [8]. MinIO provides S3-compatible object storage [9]. The CV analyzer parses PDFs and images using Python text extraction and OCR-related libraries. The job miner imports jobs from demo, API, and scraping-style sources using FastAPI, Scrapy, and HTML parsing concepts [16], [17], [18].

## 1.7 Project Scope

The scope covers a graduation/demo environment: local Docker deployment, student workflows, admin workflows, testing, screenshots, and documentation. It does not claim production readiness, job placement certainty, full job market coverage, legal compliance for production privacy, or complete AI accuracy.

## 1.8 Graduation Demo Positioning

CareerCompass should be presented as a working academic prototype with realistic engineering boundaries. The screenshots and tests show that the core workflows can run locally. However, large-scale evaluation, production identity management, cloud infrastructure, observability hardening, legal privacy review, and complete market integrations remain future work.

## 1.9 Report Organization

Chapter 2 analyzes requirements and users. Chapter 3 presents architecture, diagrams, database design, and deployment. Chapter 4 lists software and tools with references. Chapter 5 documents implementation modules from the repository. Chapter 6 presents testing and evaluation results. Chapter 7 discusses security and privacy. Chapter 8 concludes with achievements, limitations, and future work.

## Chapter 2: System Analysis

### 2.1 Introduction

System analysis defines what CareerCompass should do, who uses it, and what constraints influence implementation. The analysis is based on the reviewed repository, including the root documentation, Docker configuration, Laravel API, React frontend, Python services, database migrations, seeders, tests, and finalization notes.

### 2.2 Development Methodology

The project followed an iterative engineering approach. Each feature was implemented, checked through local commands or tests, integrated with Docker services, then documented. This approach is appropriate for a graduation project because it supports incremental progress while keeping the system demonstrable. The architecture uses separate services, which follows a common microservice-style pattern where independently deployable services communicate over network APIs [29].

### 2.3 Existing Career Guidance Problems

The system targets common problems in student career preparation:

- CV content is not structured for direct software comparison.
- Job descriptions vary in wording and completeness.
- Students need explainable feedback rather than unexplained match numbers.
- Admin users need visibility into imported jobs and scraping sources.
- Demo systems need reliable startup, testing, and evidence for academic evaluation.

### 2.4 Target Users and Stakeholders

| Stakeholder   | Description                                            | Main Interest                                                             |
|---------------|--------------------------------------------------------|---------------------------------------------------------------------------|
| Student user  | A university student or early-career user.             | Upload CV, view profile, discover jobs, analyze gaps, save opportunities. |
| Administrator | A project operator or supervisor/demo administrator.   | Review jobs, source diagnostics, users, target roles, and system health.  |
| Supervisor    | Academic supervisor evaluating the graduation project. | Correctness, completeness, originality, and honest evaluation.            |
| Project team  | Developers responsible for design and implementation.  | Maintainable code, demonstrable workflows, testing, and documentation.    |

*Table 1. Stakeholder summary.*

### 2.5 User Roles

CareerCompass implements two practical roles. The student role can register, login, upload a CV, view recommendations, run gap analysis, and track applications. The admin role can access protected admin routes for dashboard statistics, job administration, scraping sources, target roles, and user review.

## 2.6 Functional Requirements

| ID    | Requirement                            | Implementation Evidence                                                               |
|-------|----------------------------------------|---------------------------------------------------------------------------------------|
| FR-01 | Register and login users.              | Laravel AuthController, RegisterRequest, LoginRequest, React Login/Register pages.    |
| FR-02 | Upload a CV file.                      | CvUploadRequest validates PDF/JPEG/PNG and max size; Dashboard appends file field cv. |
| FR-03 | Store CV files privately.              | CvStorageService and MinIO/S3-compatible disk configuration.                          |
| FR-04 | Parse CV and extract profile/skills.   | CvProcessingService and AI CV Analyzer /api/parse-cv.                                 |
| FR-05 | Display normalized profile and skills. | UserResource, Profile page, Dashboard AI insight components.                          |
| FR-06 | Import and display jobs.               | JobPosting model, ScrapedJobController, JobController, AI Job Miner.                  |
| FR-07 | Estimate job recommendations.          | JobController and GapAnalysisService use CV/profile data and matching service.        |
| FR-08 | Analyze skill gaps.                    | GapAnalysisController and GapAnalysis page.                                           |
| FR-09 | Track applications.                    | ApplicationController, ApplicationTrackerService, Applications page.                  |
| FR-10 | Provide admin dashboards.              | Admin Dashboard, Jobs, Sources, Targets pages and admin API routes.                   |
| FR-11 | Provide health and metrics endpoints.  | HealthController, MetricsController, Prometheus/Grafana compose services.             |

*Table 2. Functional requirements summary.*

## 2.7 Non-Functional Requirements

| Category        | Requirement                                                                    | CareerCompass Approach                                                                              |
|-----------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Usability       | The UI should guide students through CV upload and recommendations.            | React pages, dashboard cards, profile score, and action buttons.                                    |
| Maintainability | Code should be modular.                                                        | Controllers, requests, services, resources, models, React pages, and Python services are separated. |
| Reliability     | The system should degrade honestly if AI services fail.                        | CV processing returns timeout/error/no_text statuses and preserves prior data when appropriate.     |
| Security        | Authentication, validation, private files, and admin role checks are required. | Sanctum tokens, request validation, admin middleware, signed URLs, service tokens.                  |
| Observability   | Health and metrics should be available.                                        | /api/health, /api/ready, /api/metrics, Prometheus, Grafana.                                         |
| Portability     | The demo should run locally.                                                   | Docker Compose services and environment examples.                                                   |

*Table 3. Non-functional requirements summary.*

## 2.8 Hardware Requirements

For local demonstration, a developer machine capable of running Docker Desktop and multiple containers is required. CV parsing and OCR-like processing can be CPU-intensive; therefore, enough memory should be available for the Laravel backend, MySQL, frontend, Python services, MinIO, Prometheus, and Grafana. GPU acceleration is not required for the demonstrated flow.

## 2.9 Software Requirements

| Layer          | Software                                                                             |
|----------------|--------------------------------------------------------------------------------------|
| Frontend       | React, Vite, Tailwind-style CSS classes, lucide-react icons, Recharts.               |
| Backend        | Laravel 12, PHP, Composer dependencies, Sanctum-style token authentication.          |
| AI services    | Python, FastAPI, text extraction, OCR-related dependencies, Scrapy-style job mining. |
| Data           | MySQL 8.0, MinIO/S3-compatible storage.                                              |
| Infrastructure | Docker, Docker Compose, Nginx, Prometheus, Grafana.                                  |
| Testing        | PHPUnit/Pest-style Laravel tests, pytest for Python, ESLint and Vite build.          |

Table 4. Hardware and software environment.

## 2.10 Input and Output Flow

Primary inputs include user account data, uploaded CV files, imported job records, target role settings, and administrator source configurations. Primary outputs include normalized user profiles, skill lists, CV analysis metadata, estimated job matches, gap reports, application records, admin statistics, health checks, and monitoring metrics.

## 2.11 Use Case Summary

The main use cases are shown in Figure 5. The system separates student and administrator responsibilities while sharing the same backend API and database.

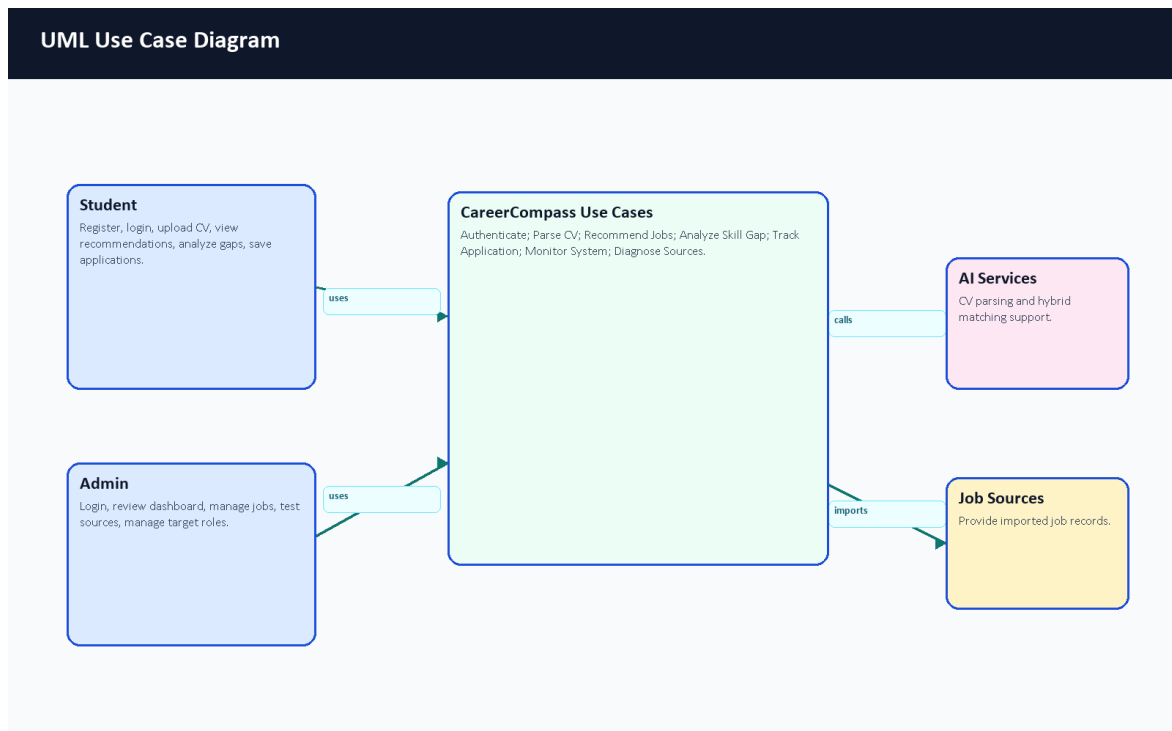


Figure 5. UML use case diagram.

## Chapter 3: System Design and Architecture

### 3.1 Introduction

CareerCompass is designed as a Dockerized multi-service application. This design separates browser UI, API logic, AI services, data storage, object storage, reverse proxy routing, and monitoring. Docker containers help package runtime dependencies consistently [10], while Docker Compose coordinates the multi-container local deployment [11].

### 3.2 High-Level System Architecture

The high-level architecture is shown in Figure 1. Browser users interact with the React frontend through Nginx. The frontend calls the Laravel API. Laravel persists records in MySQL, stores CV files in MinIO-compatible storage, calls the AI CV Analyzer for parsing, calls matching logic for recommendations/gaps, and receives job imports from the job miner.

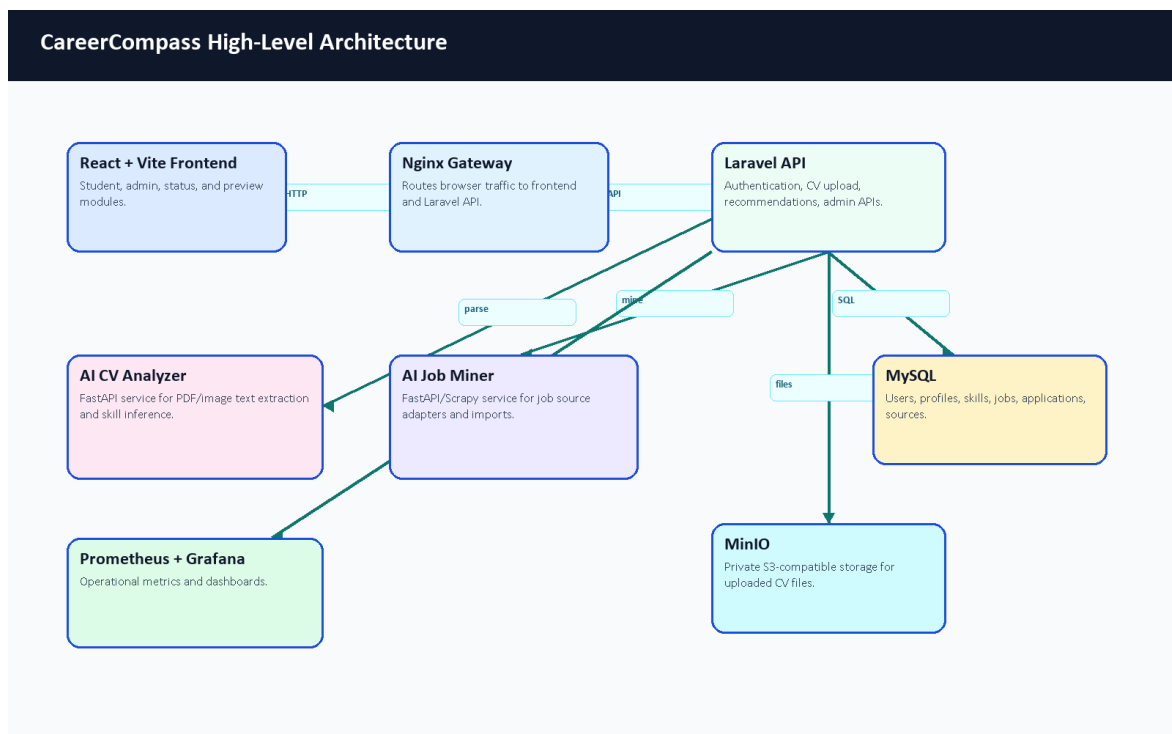


Figure 1. High-level architecture of CareerCompass.

### 3.3 Frontend Architecture

The frontend is implemented with React and organized around routes in `frontend/src/App.jsx`. Public pages include Home, Login, Register, About, Privacy, Terms, and System Status. Protected student routes include Dashboard, Jobs, Gap Analysis, Profile, Settings, Market Intelligence, Applications, CV Builder, Mock Interview, Learning, Career Planner, Mentorship, and Tools Hub. Protected admin routes include Admin Dashboard, Jobs, Users, Sources, and Target Roles.

The frontend API layer is located under `frontend/src/api`. Axios is configured in `client.js`, including base URL resolution, bearer token injection, request IDs, retry behavior for safe GET requests, and 401 handling. Authentication state is managed by `AuthContext.jsx`, which stores the user and token in local storage. Localization files exist under `frontend/src/locales`.

### 3.4 Backend API Architecture

The backend is a Laravel API. Routes are defined in `backend-api/routes/api.php` and are registered both at `/api` and `/api/v1`. The API includes public health/readiness/metrics endpoints, guest authentication routes, public job listing routes, internal scraper import routes protected by a service token, authenticated student routes, and admin routes protected by middleware.

Laravel provides structured controllers, form requests, resources, services, models, migrations, seeders, and tests. This aligns with Laravel's documented framework responsibilities, including routing, validation, database access, queues, and testing [1], [3].

### 3.5 AI CV Analyzer Architecture

The AI CV Analyzer is a FastAPI service. Laravel sends CV files to this service for parsing. The analyzer extracts readable text from PDF or image inputs, attempts structured inference, and returns fields such as predicted role, seniority, domain, skills, strengths, gaps, red flags, confidence, and parsing status. The backend handles statuses such as success, OCR fallback, timeout, error, empty file, and no text. PDF and OCR-related libraries are supported by external tools such as PyMuPDF, pdfplumber, and EasyOCR [22], [23], [24].

### 3.6 AI Job Miner Architecture

The AI Job Miner is a FastAPI service with scraping and import support. It includes source adapters for demo/local data, public APIs, and HTML/Scrapy-style extraction. Scrapy is a Python framework for extracting structured data from websites [17], and BeautifulSoup is commonly used to parse HTML documents [18]. CareerCompass uses a quality gate and honest source classifications so that demo/imported data is not overstated as complete market coverage.

### 3.7 Database Design

MySQL stores users, profiles, skills, experience records, CV analyses, job postings, applications, scraping sources, target job roles, scraping jobs, failed URLs, and related metadata. MySQL is a relational database system documented by Oracle [8]. The Laravel migrations define schema constraints, indexes, foreign keys, and unique combinations such as job title/company uniqueness.

### 3.8 ERD

Figure 8 summarizes the main database tables and relationships. It is not a complete replacement for migrations, but it provides a readable graduation-book view of the data model.

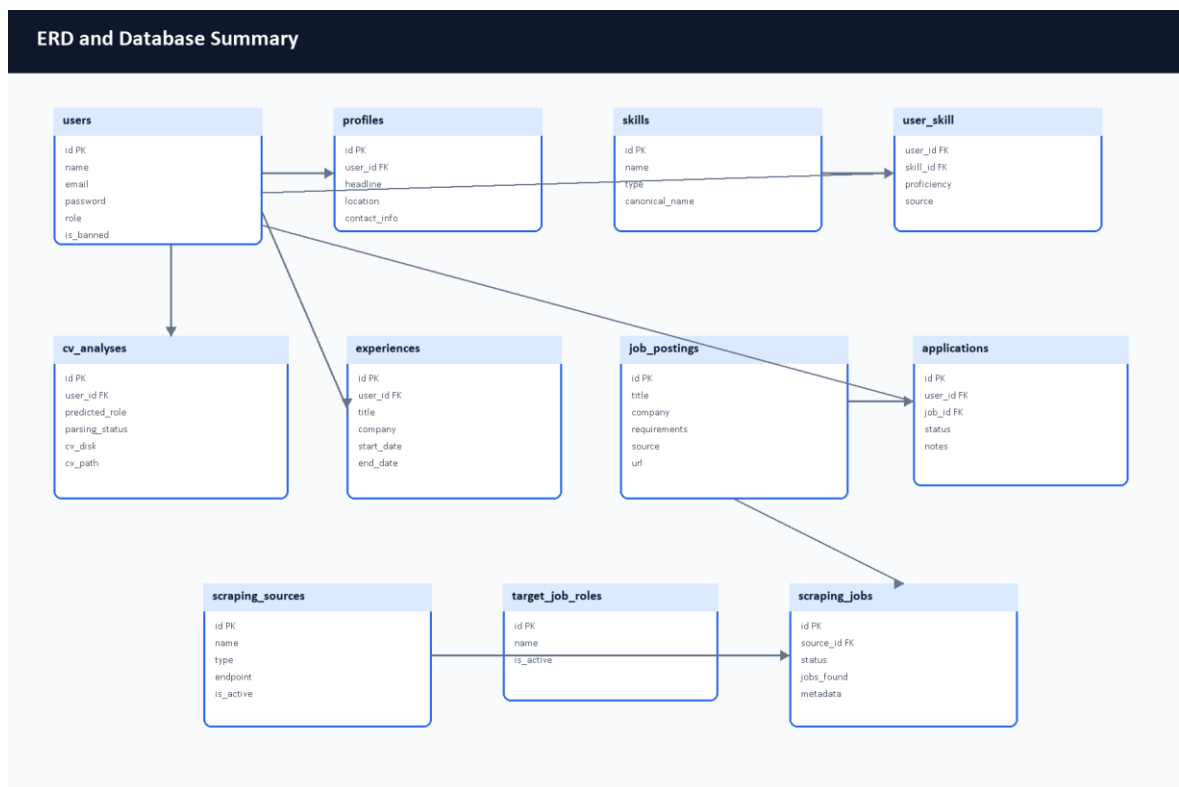


Figure 8. ERD and database summary diagram.

### 3.9 Data Flow Diagrams

The context-level data flow is shown in Figure 3, and the expanded process-level view is shown in Figure 4. Student and administrator workflows enter the same system boundary, while external job sources and AI services interact with controlled backend processes.

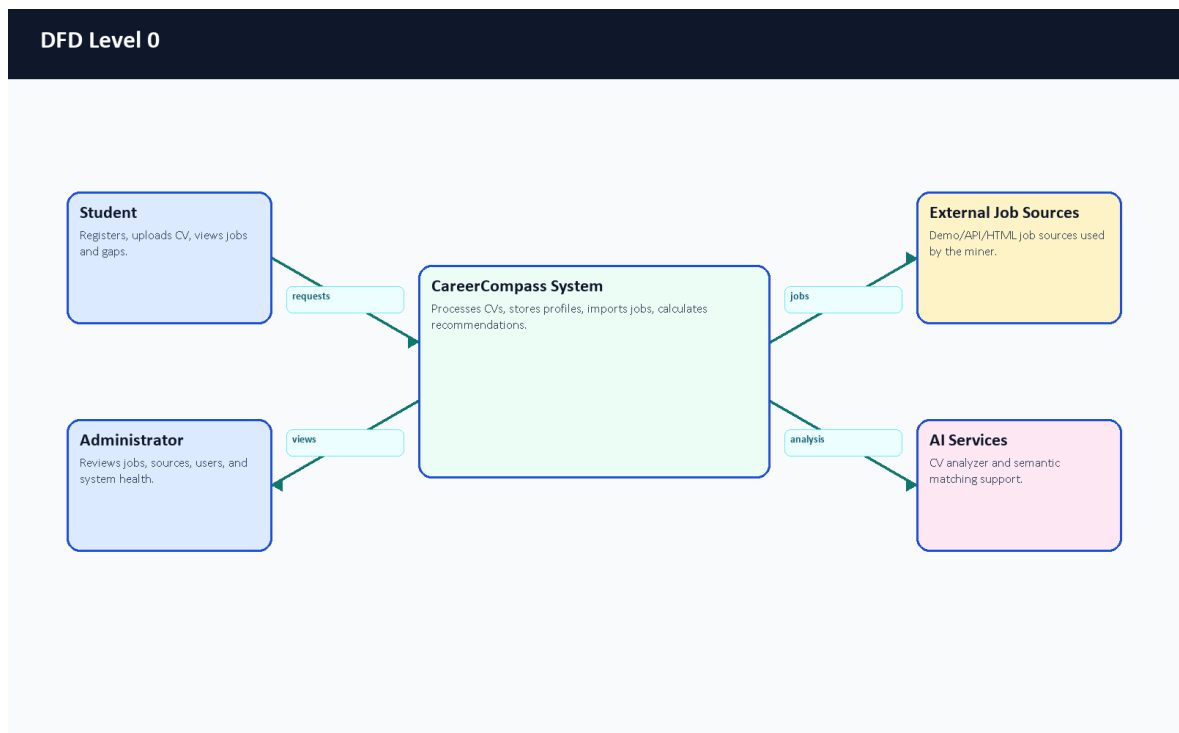


Figure 3. DFD Level 0 context diagram.

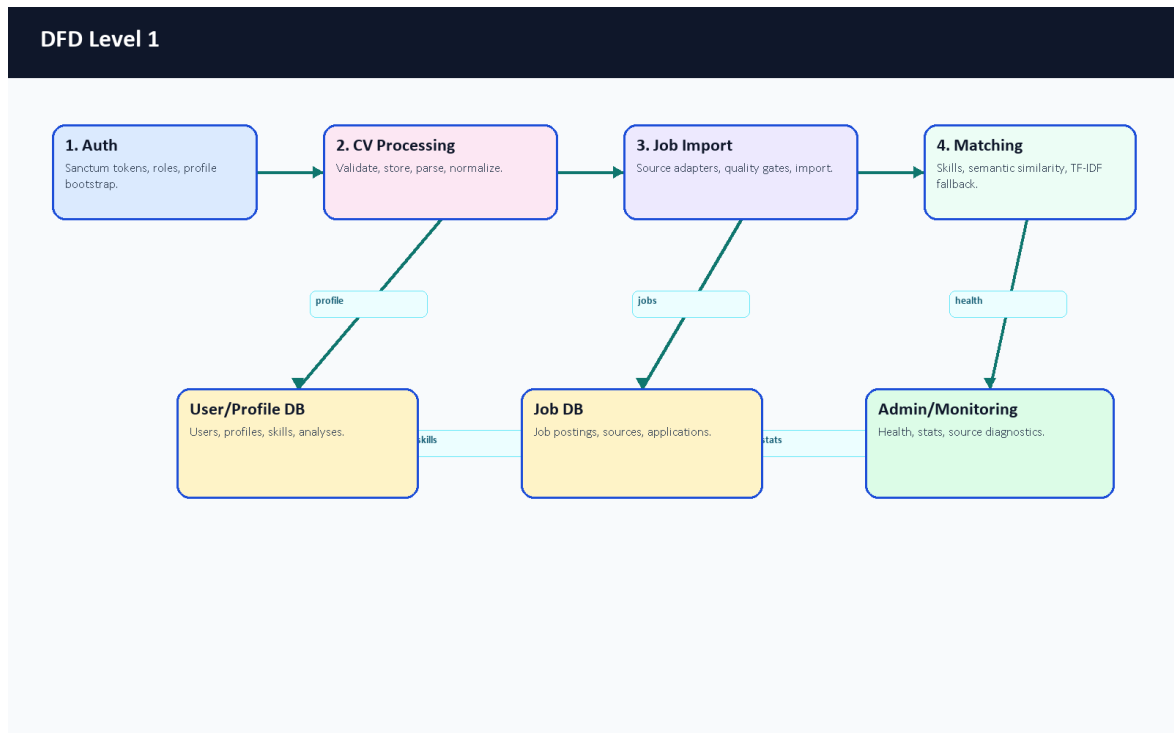


Figure 4. DFD Level 1 process diagram.

### 3.10 UML Use Case Diagram

The use case diagram separates student actions from administrator actions. Student workflows focus on career exploration. Admin workflows focus on operating and inspecting the imported job ecosystem.

### 3.11 UML Sequence Diagrams

Figure 6 shows the CV upload and analysis sequence. Figure 7 shows recommendation and gap analysis.

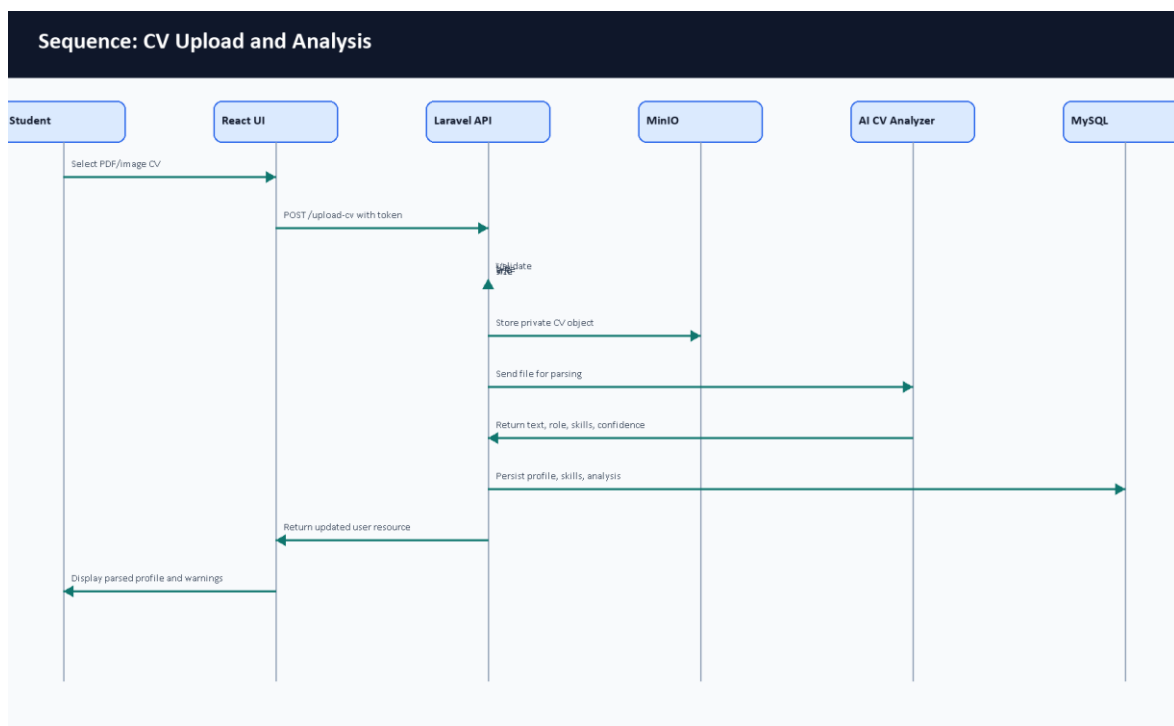


Figure 6. Sequence diagram for CV upload and analysis.

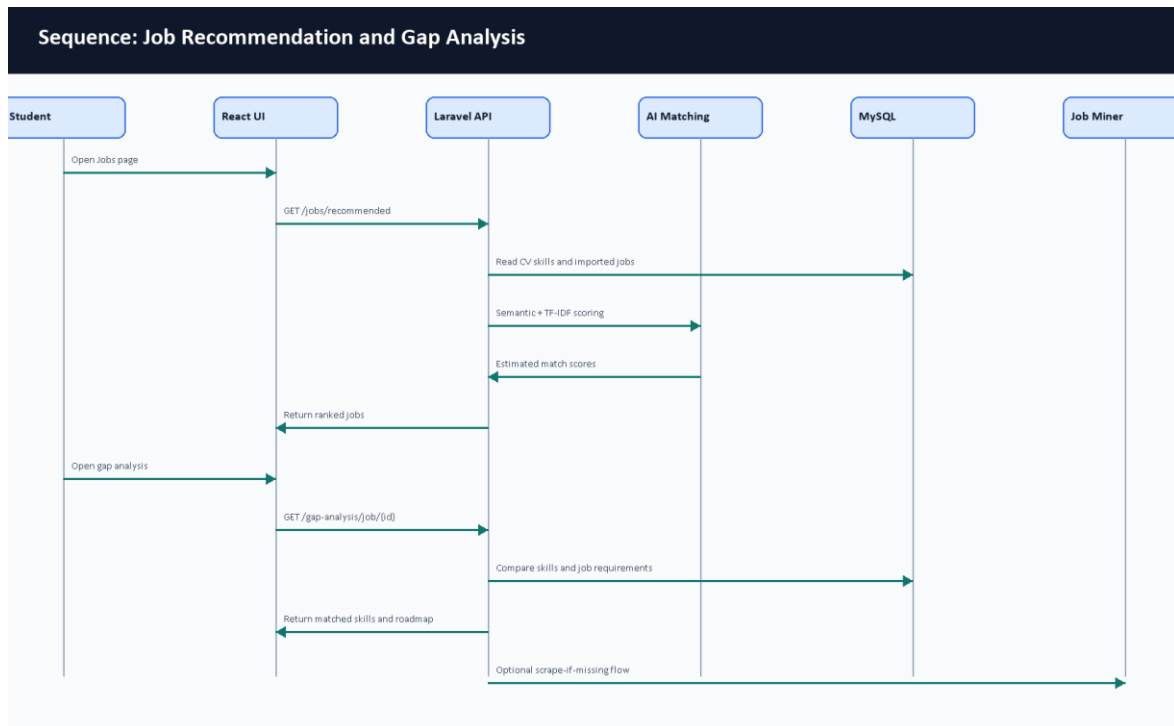


Figure 7. Sequence diagram for recommendation and gap analysis.

### 3.12 Deployment Architecture with Docker

The deployment is defined by Docker Compose files. Nginx exposes the application, the frontend serves built React assets, the Laravel API and workers handle backend work, MySQL stores structured data, MinIO stores private CV objects, Python services provide AI CV parsing and job mining, and Prometheus/Grafana provide monitoring. Figure 2 summarizes the container layout.

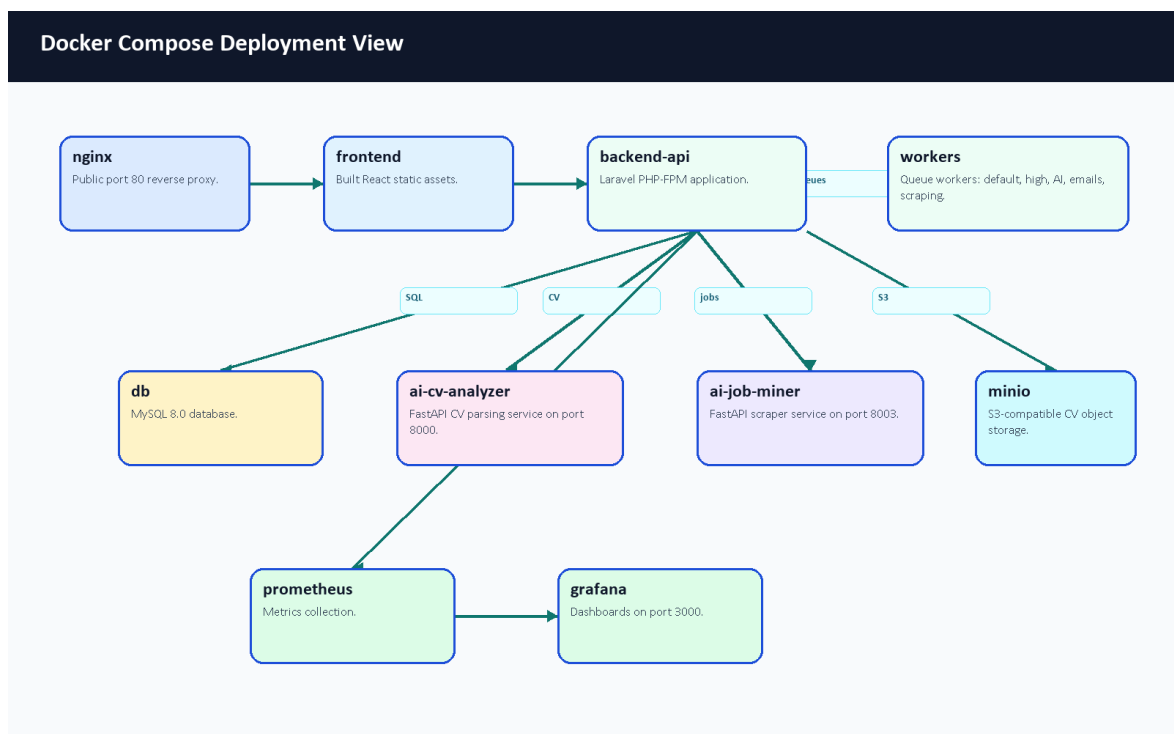


Figure 2. Docker deployment architecture.

### 3.13 Monitoring Architecture

CareerCompass includes live, readiness, and metrics endpoints. Prometheus is used for scraping and time-series metrics collection [13], while Grafana visualizes metrics and dashboard panels [14]. The admin dashboard also exposes application-level health information for the graduation demo.

### 3.14 Design Decisions and Justification

| Decision                                 | Justification                                                                                                       |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Use Laravel for the main API.            | The project benefits from built-in routing, validation, Eloquent ORM, queues, resources, tests, and middleware [1]. |
| Use React for UI.                        | Component-based pages support student/admin route separation and reusable cards [4].                                |
| Use FastAPI for AI services.             | Python AI/NLP dependencies are easier to isolate behind typed HTTP services [6].                                    |
| Use Docker Compose.                      | Multiple services can be started consistently for a graduation defense [11].                                        |
| Use private object storage for CV files. | CV files are sensitive; private storage and signed downloads reduce accidental exposure [9], [27].                  |
| Keep AI wording honest.                  | Match scores and CV parsing are estimates; the demo should avoid claiming certain outcomes.                         |

*Table 5. Design decisions summary.*

## Chapter 4: Software and Tools Used

### 4.1 Laravel

Laravel is used for the backend API, controllers, middleware, validation requests, Eloquent models, migrations, resources, queues, tests, and storage integration [1]. In CareerCompass, Laravel centralizes authenticated business logic and data persistence.

### 4.2 React

React is used to build the browser-based interface for students and administrators [4]. The project uses React routes and components for dashboard cards, forms, pages, tables, modals, and visualization panels.

### 4.3 Vite

Vite is used as the frontend build tool [5]. The successful production build transformed 2904 modules during validation.

### 4.4 FastAPI

FastAPI is used for Python services because it supports efficient API development using Python type hints and modern web service patterns [6]. CareerCompass uses it for CV analysis and job mining service endpoints.

### 4.5 Python

Python is used for AI and scraping-related services [7]. It supports the ecosystem of text extraction, OCR, NLP, testing, and scraping libraries used by the AI services.

### 4.6 MySQL

MySQL stores relational project data including users, profiles, skills, jobs, applications, scraping sources, and CV analyses [8].

### 4.7 MinIO / S3-Compatible Storage

MinIO-compatible object storage is used to store CV files privately. This avoids placing uploaded CVs directly in public web directories and supports signed temporary download flows [9].

### 4.8 Docker and Docker Compose

Docker containers package each runtime, and Docker Compose coordinates multi-container execution [10], [11]. CareerCompass uses Compose for Nginx, frontend, backend, workers, MySQL, MinIO, AI services, Prometheus, and Grafana.

### 4.9 Nginx

Nginx acts as the reverse proxy and public gateway for the local deployment [12]. It routes browser and API traffic to the correct containers.

### 4.10 Prometheus and Grafana

Prometheus collects metrics [13], and Grafana visualizes metrics and dashboards [14]. CareerCompass uses these tools to support monitoring-oriented evaluation.

## 4.11 GitHub Actions

GitHub Actions is configured for repository automation and CI/CD-style validation [15]. The report treats workflow definitions as part of the development process, while final PR status should be reviewed after the draft PR is opened.

## 4.12 NLP / AI Libraries

The project uses concepts and libraries related to text extraction, OCR, TF-IDF, cosine similarity, and sentence embeddings. TF-IDF and cosine similarity are documented by scikit-learn [19], [20]. Sentence Transformers provides sentence embedding models and utilities [21]. PyMuPDF, pdfplumber, and EasyOCR support PDF/image text extraction and OCR-style workflows [22], [23], [24].

## 4.13 Testing Tools

Backend tests use Laravel/PHP testing tools and PHPUnit concepts [25]. Python service tests use pytest where available [26]. Frontend validation uses ESLint and Vite build checks.

## 4.14 Development and Version Control Tools

Git and GitHub are used for version control and pull-request-based collaboration. Docker Desktop provides the local container runtime. Browser screenshots were captured through a Chrome DevTools Protocol helper to provide evidence images for this report.

## Chapter 5: System Implementation

### 5.1 Introduction

This chapter documents the implemented CareerCompass modules based on repository files. The implementation is not a generic career platform; it is a Laravel/React/FastAPI/Docker system with specific routes, services, pages, models, seeders, and tests.

### 5.2 Authentication and User Management

Authentication is implemented in backend-api/app/Http/Controllers/Api/AuthController.php. Registration creates a user with role user, loads profile and related resources, and returns a token. Login validates credentials, checks the banned state, revokes old tokens, creates a new token, and returns a user resource. The frontend stores the token as auth\_token and the user object in local storage through frontend/src/context/AuthContext.jsx.

The register request restricts emails to selected public email domains and validates password format. The admin role is protected by the IsAdmin middleware. Admin seed data creates a demo-only administrator account through AdminUserSeeder.

### 5.3 Student Dashboard

The student dashboard is implemented in frontend/src/pages/user/Dashboard.jsx. It presents the current profile state, CV upload/update controls, profile completeness, career identity, AI insights, and next actions. Before CV upload, it prompts the user to add a CV. After upload, it displays parsed CV availability, role inference, profile score, experience, and action buttons.

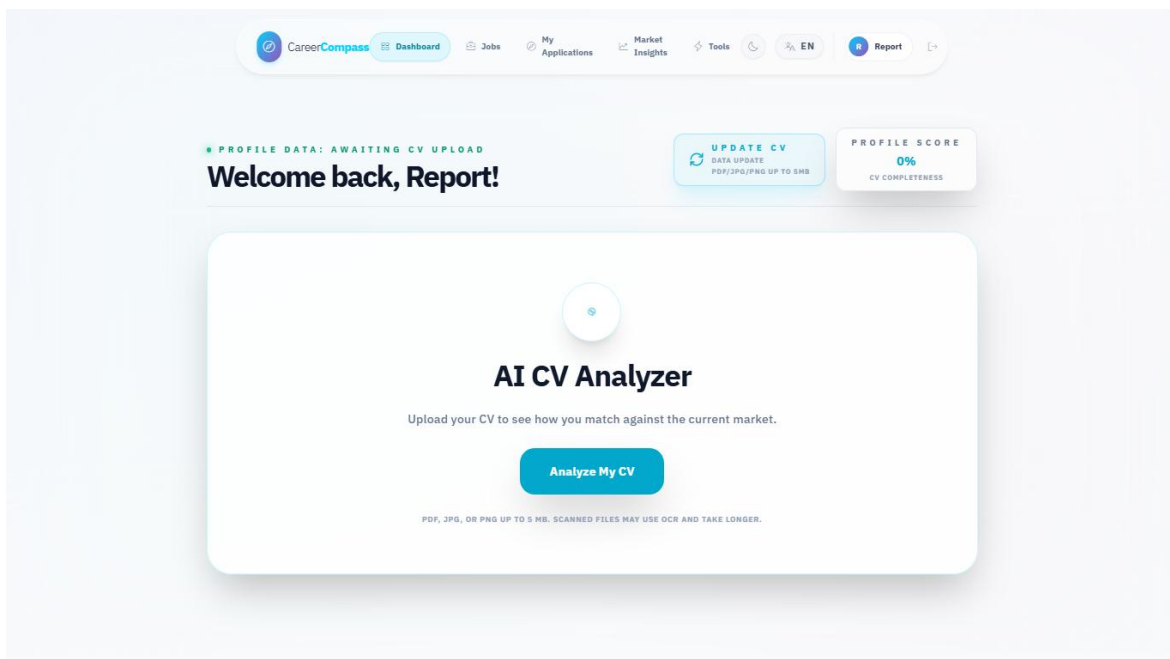


Figure 12. Student dashboard before CV upload.

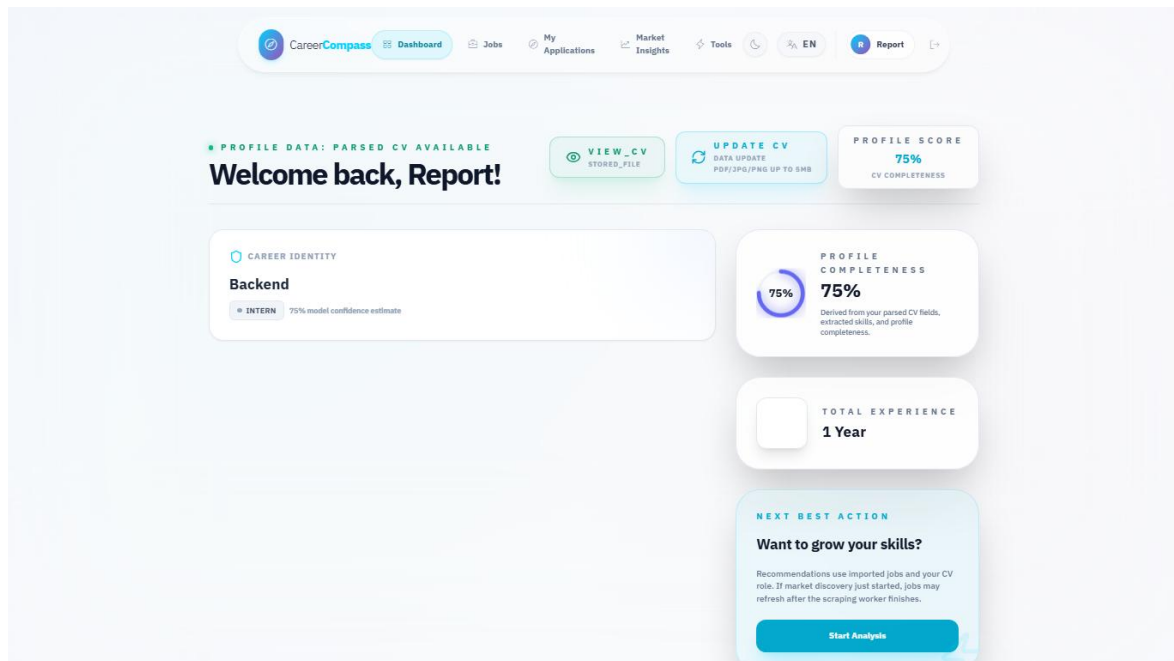


Figure 14. Dashboard after successful CV parsing.

## 5.4 CV Upload and Storage

CvUploadRequest requires a cv file and accepts PDF, JPEG, JPG, and PNG files up to 5 MB. The frontend appends the selected file as cv in a FormData object. CvController calls the CV processing service, persists the file path and metadata, and returns a unified user resource.

CV storage is handled as a private file workflow. The system supports signed download URLs, which is a better demo posture than public file exposure. OWASP recommends validating uploaded file type, extension, size, and storage handling carefully [27].

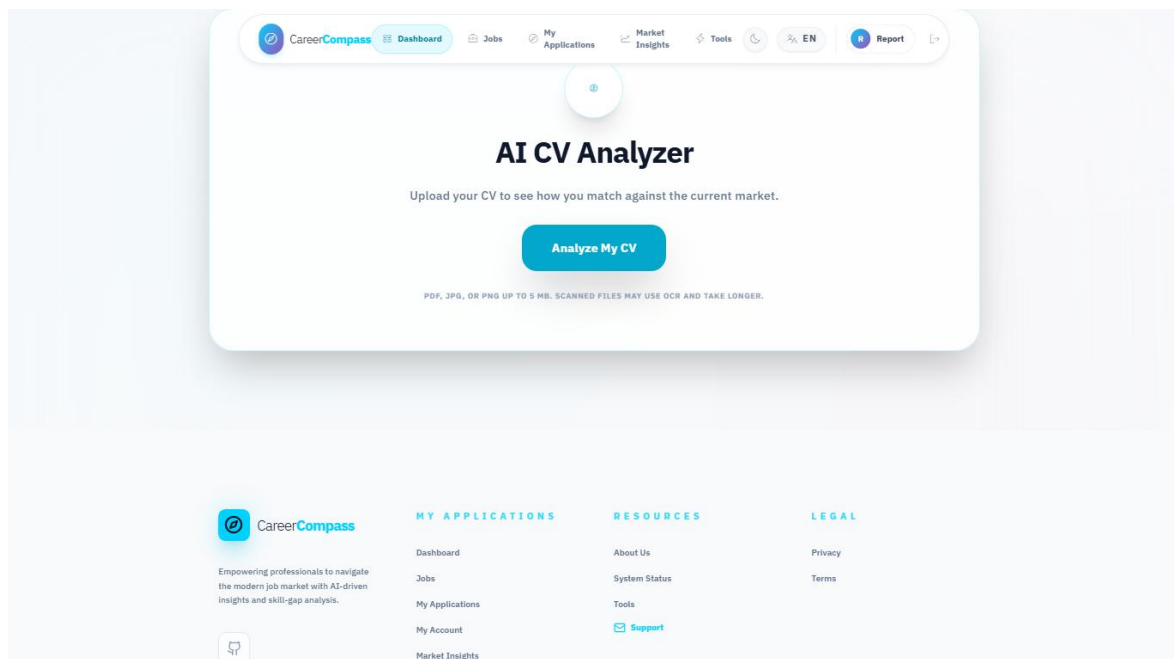


Figure 13. CV upload user interface.

## 5.5 CV Parsing and Skill Extraction

The CV processing flow sends the file to the AI CV Analyzer, receives parsed data, synchronizes skills, updates profile fields, and stores CV analysis metadata. The implementation handles multiple parsing statuses honestly. If analysis times out, fails, or finds no readable text, the backend returns warnings and preserves existing profile details rather than silently replacing data with low-quality output.

## 5.6 Profile and Skills Management

The profile page reads normalized user data, profile fields, experiences, skills, and CV analysis. The system distinguishes user fields, profile fields, extracted skills, predicted role, seniority, and completeness score. Skill synchronization is handled through backend services rather than only frontend state.

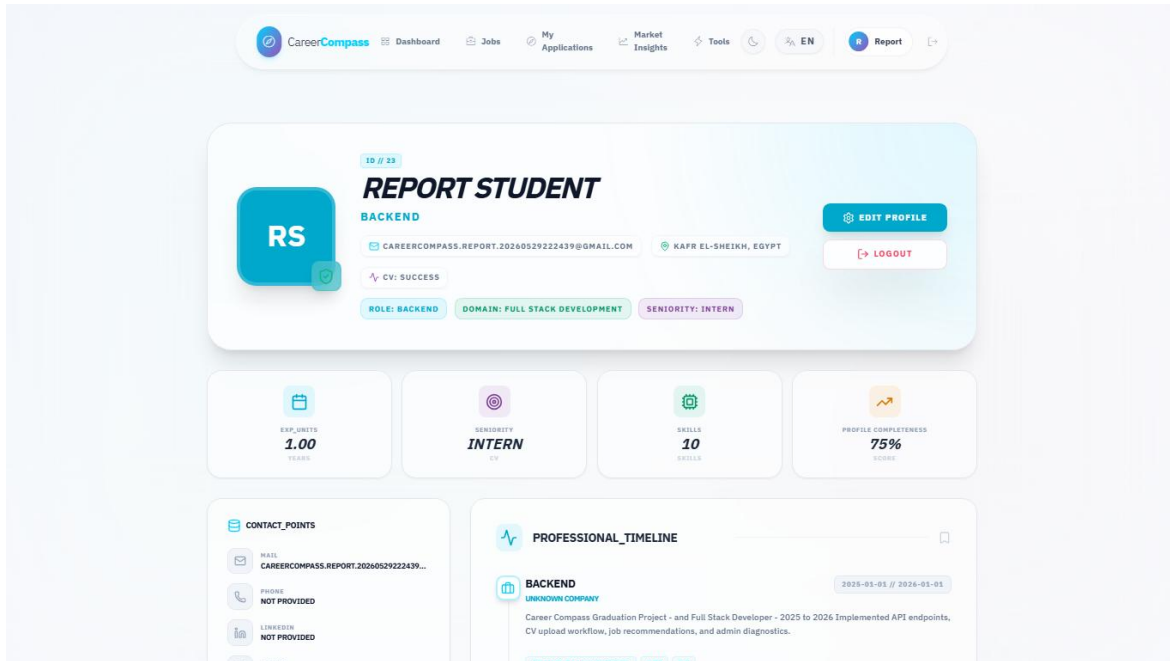


Figure 15. Extracted profile and skills page.

## 5.7 Job Data Model

Jobs are represented in the backend through job posting models and migrations. Fields include title, company, description/requirements, URL, source, and metadata. The seeders and import controllers enforce quality gates and uniqueness rules, including a title/company uniqueness constraint that prevented duplicate seed insertion during validation.

## 5.8 AI Job Miner and Scraping Sources

The job miner exposes a FastAPI service and imports jobs using configured sources. The backend protects scraper import routes with an internal service token. Admin pages expose source diagnostics, source status, testing, and target role management. The project differentiates demo/local sources, API sources, and HTML/scraping sources instead of claiming complete market coverage.

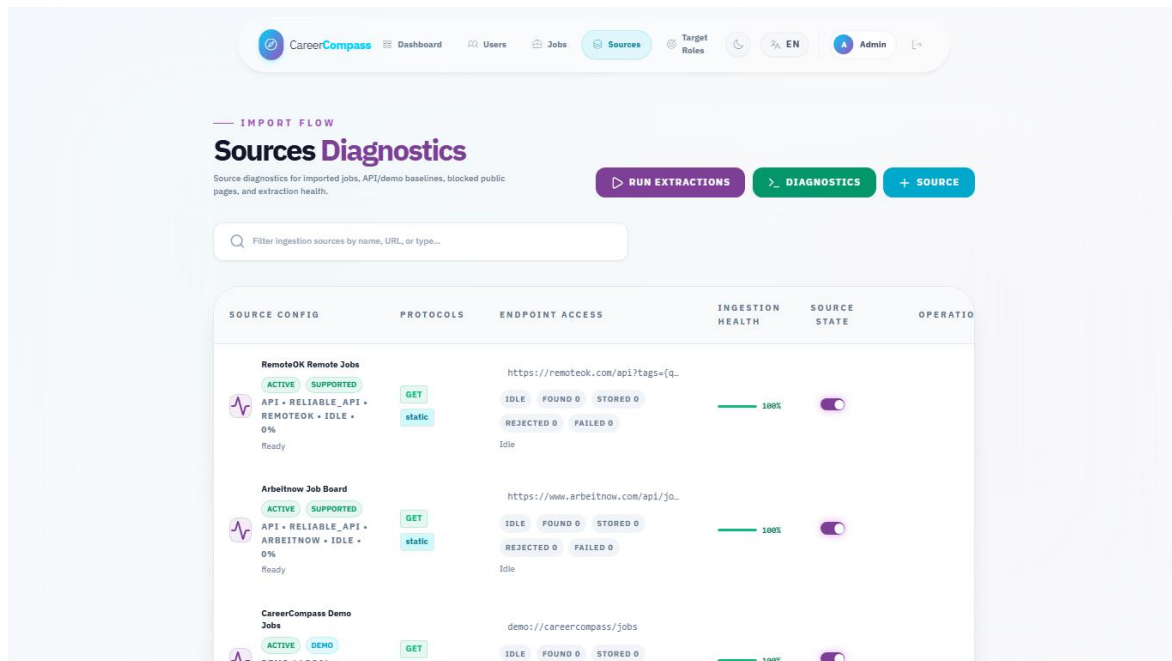


Figure 24. Admin sources diagnostics page.

## 5.9 Job Recommendations

The jobs page requests recommended jobs when no manual search query is active. Recommendations are based on CV/profile context when available. Matching combines normalized database data with semantic and TF-IDF-style comparison where available. TF-IDF represents text using term frequency and inverse document frequency weighting [19], while cosine similarity compares vector orientation [20].

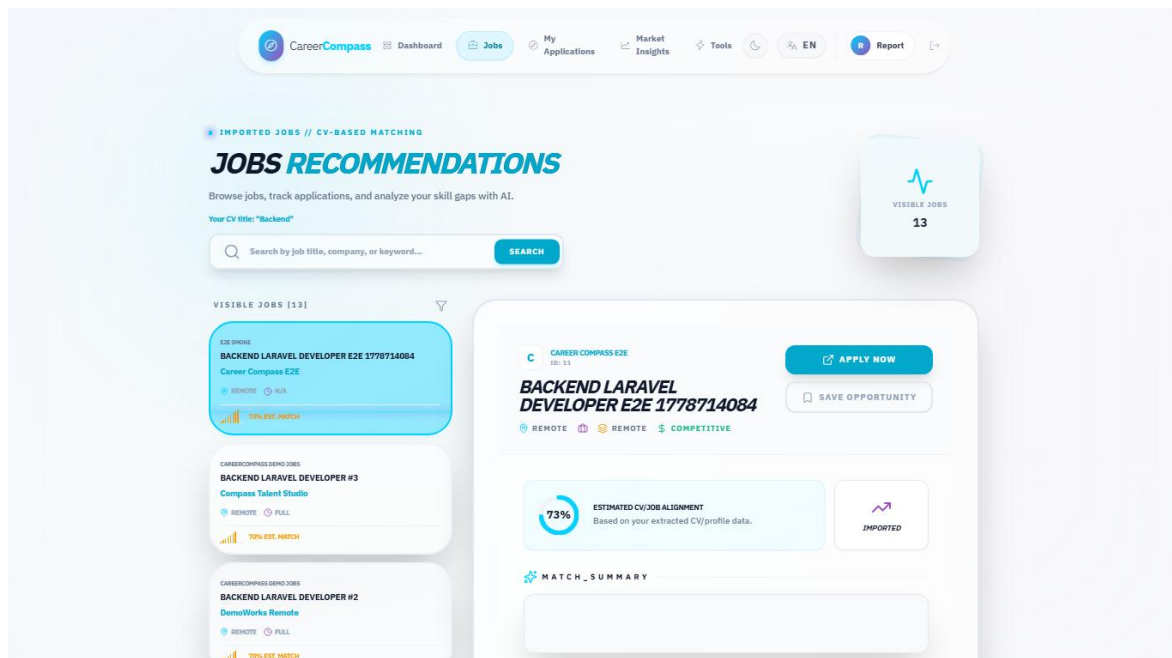


Figure 16. Jobs recommendations page.

## 5.10 Gap Analysis

Gap analysis compares a selected job or target role against the user's profile and extracted skills. It returns matched skills, critical/missing skills, recommendations, match percentage, and roadmap-like guidance. The frontend displays these outputs in an explainable layout rather than a single opaque score.

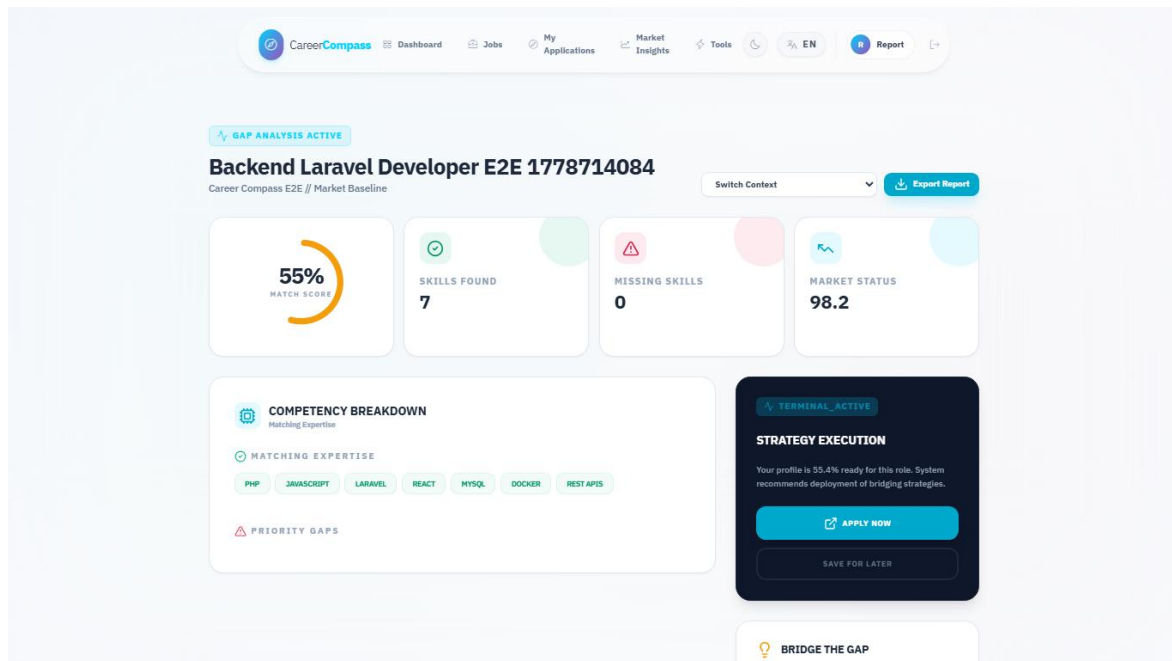


Figure 18. Gap analysis page.

## 5.11 Application Tracker

The application tracker is implemented through ApplicationController, ApplicationTrackerService, and frontend/src/pages/user/Applications.jsx. Students can save a job, update status, view counts, and delete tracked items. The backend validates job existence and allowed statuses.

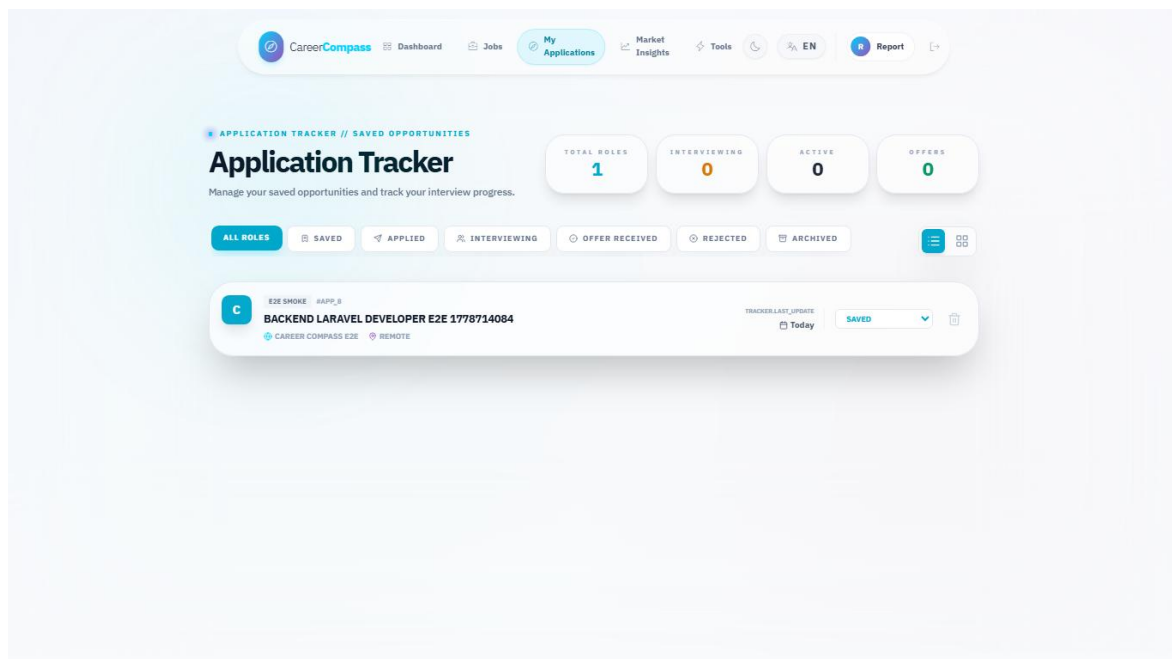


Figure 19. Applications tracker page.

## 5.12 Admin Dashboard

The admin dashboard summarizes users, imported jobs, active sources, target roles, health status, and scraping batch progress. It is protected by admin middleware and uses admin API routes.

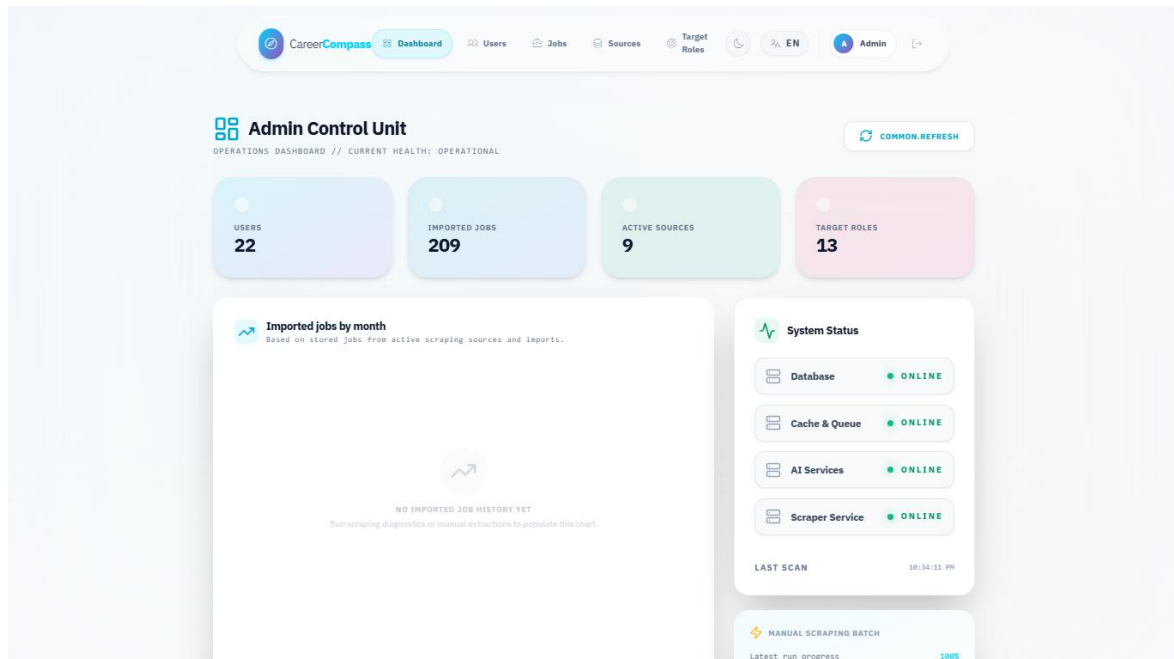


Figure 22. Admin dashboard.

## 5.13 Admin Source Diagnostics

The source diagnostics page lists configured scraping sources, supports source testing, and displays quality and scraping status information. The target roles page manages role names used by scraping and market discovery.

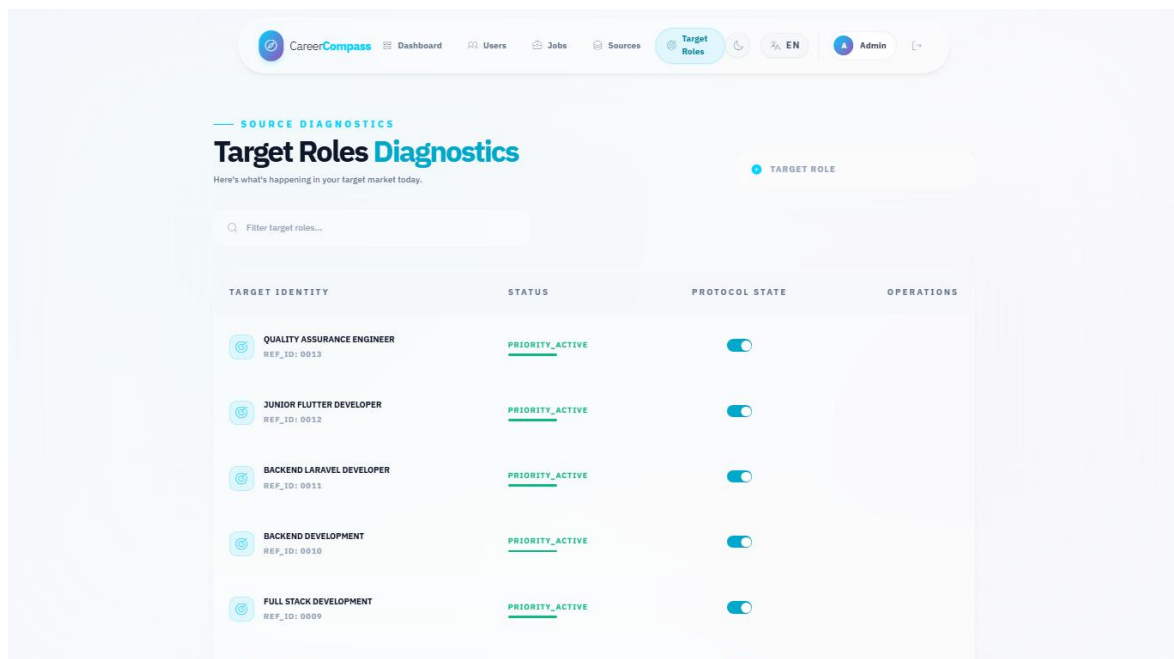


Figure 25. Admin target roles page.

## 5.14 System Health and Monitoring

Health endpoints include live and readiness checks. The system status page presents service state to users, while admin health data supports operational monitoring. Metrics are available for Prometheus and dashboards are available through Grafana.

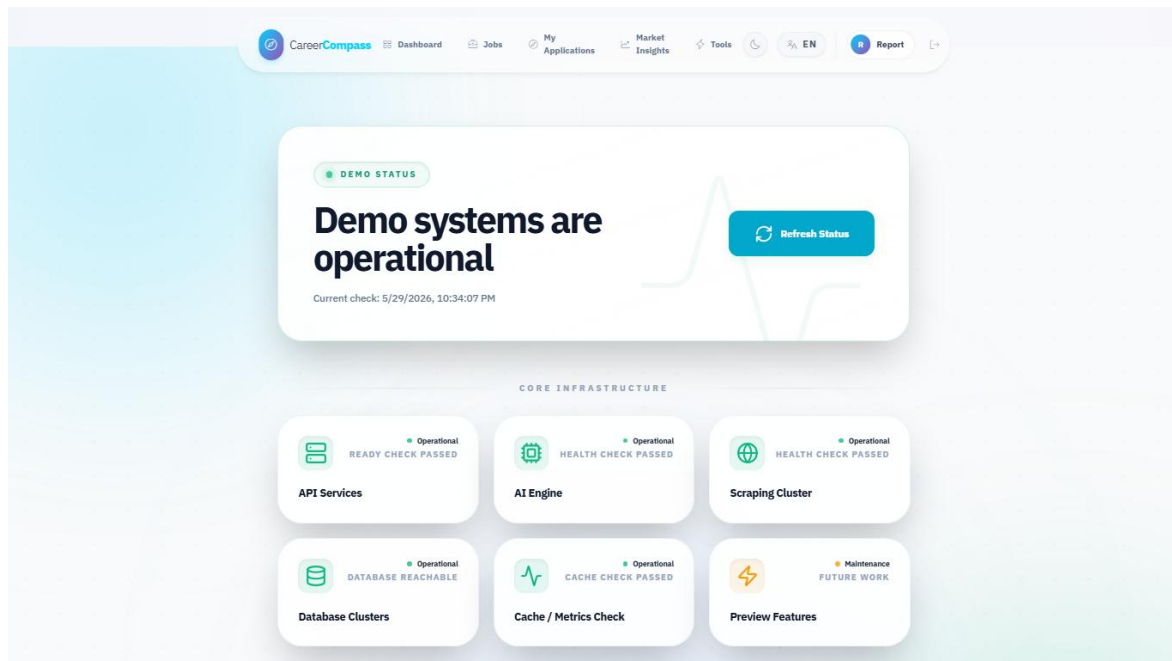


Figure 21. System status page.

## 5.15 Error Handling and Fallbacks

The code includes explicit handling for CV processing failures, AI gateway connection failures, validation errors, missing user data, empty job data, and unavailable services. The job recommendation and gap analysis code includes fallback behavior when AI services are not available.

## 5.16 Internationalization and UI Preview Modules

The frontend contains English and Arabic locale files. Preview modules include CV Builder, Mock Interview, Learning Paths, Career Planner, Mentorship, Tools Hub, and Market Intelligence. The report treats these as preview modules unless tests or implementation prove production completeness.

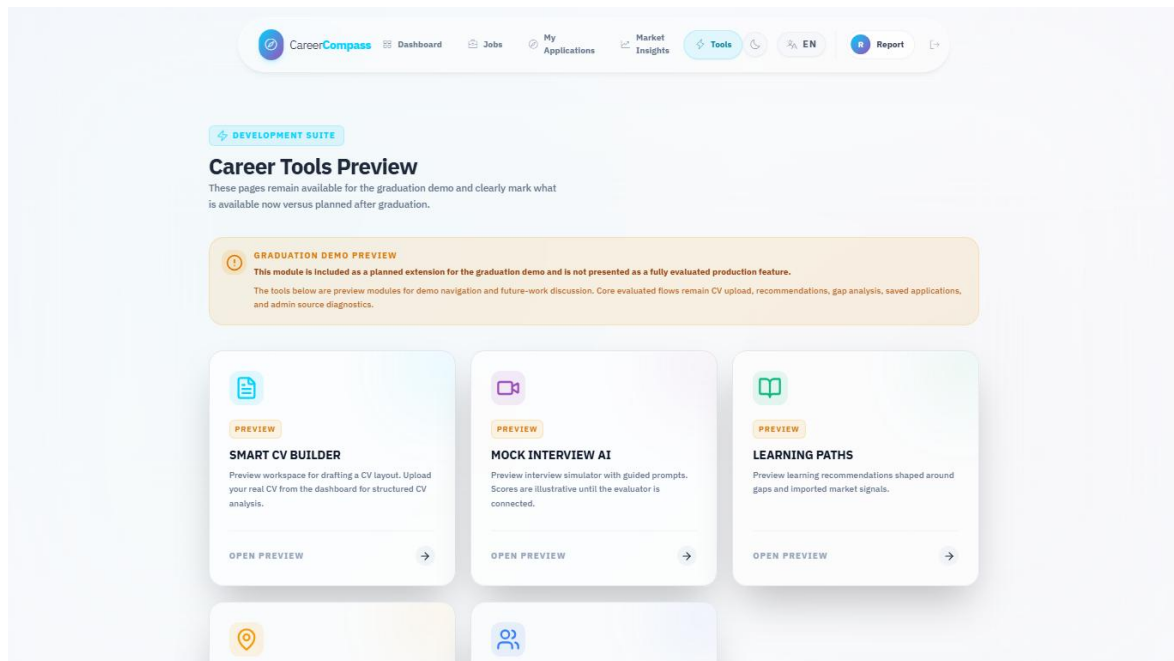


Figure 20. Tools Hub preview page.

## 5.17 Dockerized Runtime Flow

The runtime starts through Docker Compose. Nginx exposes the app, frontend and backend containers serve UI/API flows, backend workers process queues, Python services support AI workflows, MySQL and MinIO persist state, and monitoring services observe the stack.

| Docker Compose Services Evidence |                                            |                           |                         |              |  |
|----------------------------------|--------------------------------------------|---------------------------|-------------------------|--------------|--|
| NAME                             | IMAGE                                      | COMMAND                   | SERVICE                 | CREATED      |  |
| cc-backend                       | graduation-project-backend-api             | "/usr/local/bin/cc-eâ€¦"  | backend-api             | 18 hours ago |  |
| cc-backend-scheduler             | graduation-project-backend-scheduler       | "/usr/local/bin/cc-eâ€¦"  | backend-scheduler       | 18 hours ago |  |
| cc-backend-worker                | graduation-project-backend-worker          | "/usr/local/bin/cc-eâ€¦"  | backend-worker          | 18 hours ago |  |
| cc-backend-worker-ai             | graduation-project-backend-worker-ai       | "/usr/local/bin/cc-eâ€¦"  | backend-worker-ai       | 18 hours ago |  |
| cc-backend-worker-emails         | graduation-project-backend-worker-emails   | "/usr/local/bin/cc-eâ€¦"  | backend-worker-emails   | 18 hours ago |  |
| cc-backend-worker-high           | graduation-project-backend-worker-high     | "/usr/local/bin/cc-eâ€¦"  | backend-worker-high     | 18 hours ago |  |
| cc-backend-worker-scraping       | graduation-project-backend-worker-scraping | "/usr/local/bin/cc-eâ€¦"  | backend-worker-scraping | 18 hours ago |  |
| cc-cv-analyzer                   | graduation-project-ai-cv-analyzer          | "uvicorn main:app --â€¦"  | ai-cv-analyzer          | 18 hours ago |  |
| cc-db                            | mysql:8.0                                  | "docker-entrypoint.sâ€¦"  | db                      | 2 weeks ago  |  |
| cc-frontend                      | graduation-project-frontend                | "/docker-entrypoint.â€¦"  | frontend                | 18 hours ago |  |
| cc-job-miner                     | graduation-project-ai-job-miner            | "uvicorn service_apiâ€¦"  | ai-job-miner            | 18 hours ago |  |
| cc-nginx                         | nginx:stable-alpine                        | "/docker-entrypoint.â€¦"  | nginx                   | 6 days ago   |  |
| graduation-project-grafana-1     | grafana/grafana-oss:11.4.0                 | "/run.sh"                 | grafana                 | 2 weeks ago  |  |
| graduation-project-minio-1       | minio/minio:latest                         | "/usr/bin/docker-entâ€¦"  | minio                   | 2 weeks ago  |  |
| graduation-project-prometheus-1  | prom/prometheus:v2.55.1                    | "/bin/sh -c 'sed \"s/â€¦" | prometheus              | 2 weeks ago  |  |

Figure 26. Docker services evidence.

## Chapter 6: Testing and Evaluation

### 6.1 Introduction

Evaluation was performed using repository-aware commands and browser evidence. The goal was to verify that each major part of the graduation/demo system runs and to document limitations honestly.

### 6.2 Testing Strategy

The testing strategy combined automated tests, build checks, configuration checks, service health probes, and manual functional evaluation. Automated tests provide repeatable evidence. Screenshots provide visible workflow evidence. Manual tables document behavior that is difficult to fully automate in the available environment.

### 6.3 Backend Testing

Backend validation was executed inside the backend container. Composer dependencies were already installed. `php artisan config:clear`, `php artisan route:list`, migrations, and tests passed. The route list confirmed 131 routes. The Laravel test suite passed with 39 tests and 297 assertions.

### 6.4 Frontend Testing

The frontend was validated using the existing frontend/node\_modules and the bundled Node runtime. ESLint passed with 9 warnings and 0 errors. The warnings were related to React fast-refresh export conventions and hook dependency notes. The Vite production build passed and transformed 2904 modules.

### 6.5 Python Services Testing

The AI Job Miner pytest suite passed with 75 tests. Python syntax compilation passed for both AI services. The AI CV Analyzer pytest command could not run because pytest was not installed in that container; this is recorded as a skipped/blocked validation step rather than a failure of the application code.

### 6.6 Docker and Integration Testing

Docker Compose configuration validation passed for both development and production overlay configurations. A full compose build/start was attempted; the initial full build exceeded the 15-minute command timeout, but the stack continued building and was later brought up successfully with targeted frontend/Nginx rebuild/start. All main containers reached healthy or running state during final checks.

### Validation Command Evidence

Validation summary captured for the graduation book:

```
docker compose config --quiet: PASSED
docker compose -f docker-compose.yml -f docker-compose.prod.yml config --quiet: PASSED
docker compose up -d --build: stack built; full initial build exceeded 15 minutes, then targeted rebuild/start completed
composer install in backend-api container: PASSED
php artisan config:clear: PASSED
php artisan route:list: PASSED (131 routes)
php artisan migrate --force --no-interaction: PASSED (nothing to migrate)
php artisan test: PASSED (39 tests, 297 assertions)
frontend ESLint: PASSED with 9 warnings and 0 errors
frontend Vite build: PASSED (2904 modules transformed)
ai-job-miner pytest: PASSED (75 passed)
ai-cv-analyzer compileall: PASSED
ai-job-miner compileall: PASSED
ai-cv-analyzer pytest: SKIPPED/blocked because pytest is not installed in that container
HTTP probes: /, /api/health, /api/ready, /status, AI CV Analyzer root, and Job Miner health returned 200
```

*Figure 27. Validation command evidence.*

## 6.7 CI/CD Validation

GitHub Actions workflow files were reviewed as part of repository inspection. A live GitHub Actions status screenshot was not captured before the draft PR because PR checks only become meaningful after the branch is pushed and GitHub schedules workflows. The manual review checklist asks the team to inspect CI status on the opened draft PR.

## 6.8 CV Analyzer Mini Dataset Evaluation

A sample PDF CV was generated for the screenshot workflow and uploaded through the running system. The upload succeeded, and the dashboard showed parsed CV data, backend role inference, extracted skills, and profile completeness. To strengthen the evaluation beyond that smoke test, this revision adds a mini synthetic dataset under docs/graduation-book/evaluation/.

The mini CV evaluation is explicitly offline and deterministic. It uses fake CV text, expected skill labels, and a keyword/role inference evaluator. It does not claim live model accuracy. The live AI CV Analyzer endpoint can be added to this mini-evaluation later, but the current document records only metrics that were actually computed from the synthetic dataset.

## 6.9 Recommendation Mini Dataset Evaluation

The recommendation mini evaluation ranks synthetic jobs for each synthetic CV using skill overlap plus domain and seniority bonuses. This validates the recommendation concept and provides a repeatable regression check for report evidence. It is not a production recommender benchmark, and the report does not claim complete job-market coverage.

## 6.10 Gap Analysis Mini Dataset Evaluation

The gap-analysis mini evaluation compares expected matched and missing skills with computed matched and missing skills for selected CV/job pairs. This directly validates the explanation structure used by the gap-analysis workflow: matched skills should be shown separately from missing skills.

## Mini Dataset Files

The mini evaluation uses fake synthetic CVs and fake synthetic job records stored under docs/graduation-book/evaluation/. It is intentionally small and preliminary. It is useful for graduation validation and regression checks, but it is not statistically representative and should not be used as a production benchmark.

| Sample ID          | Expected Role             | Seniority | Domain            | Expected Skills                                         |
|--------------------|---------------------------|-----------|-------------------|---------------------------------------------------------|
| cv_backend_laravel | Backend Laravel Developer | junior    | backend_web       | PHP, Laravel, MySQL, REST API, Docker, Git              |
| cv_frontend_react  | Frontend React Developer  | intern    | frontend_web      | React, JavaScript, Vite, HTML, CSS, API integration     |
| cv_data_ml         | Data and ML Student       | student   | data_ml           | Python, pandas, scikit-learn, NLP, data analysis        |
| cv_full_stack      | Full Stack Developer      | junior    | full_stack_web    | Laravel, React, MySQL, Docker, REST API, Git            |
| cv_qa_testing      | QA Testing Engineer       | intern    | quality_assurance | testing, test cases, pytest, API testing, bug reporting |

Table 6. Mini CV dataset.

| Job ID              | Title                            | Domain            | Required Skills                                         |
|---------------------|----------------------------------|-------------------|---------------------------------------------------------|
| job_laravel_backend | Junior Laravel Backend Developer | backend_web       | PHP, Laravel, MySQL, REST API, Docker, Git              |
| job_react_frontend  | React Frontend Intern            | frontend_web      | React, JavaScript, Vite, HTML, CSS, API integration     |
| job_full_stack_web  | Full Stack Web Developer         | full_stack_web    | Laravel, React, MySQL, Docker, REST API, Git            |
| job_data_analyst    | Junior Data Analyst              | data_ml           | Python, pandas, scikit-learn, NLP, data analysis        |
| job_qa_intern       | QA Automation Intern             | quality_assurance | testing, test cases, pytest, API testing, bug reporting |
| job_devops_docker   | DevOps Docker Intern             | devops            | Docker, Git, CI, Linux, monitoring                      |
| job_php_api         | PHP API Developer                | backend_web       | PHP, Laravel, REST API, MySQL, Git, Docker              |
| job_nlp_assistant   | NLP Assistant Intern             | data_ml           | Python, NLP, scikit-learn, data analysis, testing       |

Table 7. Mini job dataset.

## Metric Definitions

Skill precision measures how many extracted skills are expected labels. Skill recall measures how many expected skills were extracted. Skill F1 is the harmonic mean of precision and recall [30].

Recommendation top-1 and top-3 relevance compare ranked jobs against manual relevance labels. Gap agreement compares computed matched/missing skills against expected matched/missing skills.

| Area       | Metric                | Value | Notes                                     |
|------------|-----------------------|-------|-------------------------------------------|
| CV offline | Macro skill precision | 1.000 | Keyword extraction over synthetic CV text |

| Area                   | Metric                     | Value | Notes                                               |
|------------------------|----------------------------|-------|-----------------------------------------------------|
| CV offline             | Macro skill recall         | 1.000 | Compared with expected skill labels                 |
| CV offline             | Macro skill F1             | 1.000 | F1 computed from precision/recall [30]              |
| CV offline             | Role match rate            | 1.000 | Rule-based role inference on synthetic data         |
| CV offline             | Seniority match rate       | 1.000 | Rule-based seniority inference                      |
| CV offline             | Domain match rate          | 1.000 | Rule-based domain inference                         |
| Recommendation offline | Top-1 relevance            | 1.000 | Top recommendation belongs to manual relevant set   |
| Recommendation offline | Top-3 relevance            | 1.000 | Any top-3 job belongs to manual relevant set        |
| Recommendation offline | Mean precision@3           | 0.800 | Relevant jobs among top three                       |
| Gap offline            | Matched skill agreement F1 | 1.000 | Computed matched skills vs. expected matched skills |
| Gap offline            | Missing skill agreement F1 | 1.000 | Computed missing skills vs. expected missing skills |

Table 8. Mini evaluation metrics.

### Recommendation Ranking Details

| CV Sample          | Expected Relevant Jobs                                                                               | Top 3 Offline Recommendations                                  | P@3   |
|--------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|-------|
| cv_backend_laravel | job_laravel_backend,<br>job_php_api, job_full_stack_web,<br>job_devops_docker                        | job_laravel_backend,<br>job_php_api,<br>job_full_stack_web     | 1.000 |
| cv_frontend_react  | job_react_frontend,<br>job_full_stack_web                                                            | job_react_frontend,<br>job_full_stack_web,<br>job_qa_intern    | 0.667 |
| cv_data_ml         | job_data_analyst,<br>job_nlp_assistant                                                               | job_data_analyst,<br>job_nlp_assistant,<br>job_laravel_backend | 0.667 |
| cv_full_stack      | job_full_stack_web,<br>job_laravel_backend,<br>job_php_api, job_react_frontend,<br>job_devops_docker | job_full_stack_web,<br>job_laravel_backend,<br>job_php_api     | 1.000 |
| cv_qa_testing      | job_qa_intern, job_nlp_assistant                                                                     | job_qa_intern,<br>job_nlp_assistant,<br>job_react_frontend     | 0.667 |

Table 9. Recommendation ranking details.

### Gap Analysis Pair Details

| CV / Job Pair                                   | Matched Skills                                | Missing Skills                           | Agreement                             |
|-------------------------------------------------|-----------------------------------------------|------------------------------------------|---------------------------------------|
| cv_backend_laravel<br>-><br>job_laravel_backend | Docker, Git, Laravel, MySQL, PHP,<br>REST API | None                                     | matched F1=1.000; missing<br>F1=1.000 |
| cv_frontend_react -<br>><br>job_full_stack_web  | React                                         | Docker, Git, Laravel, MySQL,<br>REST API | matched F1=1.000; missing<br>F1=1.000 |

| CV / Job Pair                          | Matched Skills                                          | Missing Skills                               | Agreement                          |
|----------------------------------------|---------------------------------------------------------|----------------------------------------------|------------------------------------|
| cv_data_ml -><br>job_nlp_assistant     | NLP, Python, data analysis, scikit-learn                | testing                                      | matched F1=1.000; missing F1=1.000 |
| cv_qa_testing -><br>job_qa_intern      | API testing, bug reporting, pytest, test cases, testing | None                                         | matched F1=1.000; missing F1=1.000 |
| cv_full_stack -><br>job_react_frontend | React                                                   | API integration, CSS, HTML, JavaScript, Vite | matched F1=1.000; missing F1=1.000 |

Table 10. Gap analysis pair details.

## 6.11 Job Miner Evaluation

The AI Job Miner test suite passed with 75 tests. Admin source diagnostics displayed active sources and source state. The job database contained imported/demo jobs visible in the admin dashboard. Because external job sources can change or throttle scraping, long-term data quality evaluation should be repeated near the final defense.

## 6.12 Application Tracker Evaluation

The application tracker was evaluated by saving a selected job to the tracker and loading the Applications page. The screenshot shows saved opportunity state. Backend tests also include application tracker behavior.

## 6.13 Admin Dashboard Evaluation

Admin login and admin dashboard access were tested with the demo admin account. The admin dashboard, admin jobs, admin sources, and admin target roles pages were captured. The dashboard displayed 22 users, 209 imported jobs, 9 active sources, and 13 target roles at capture time.

## 6.14 Performance Observations

The local Docker stack is heavy because it runs frontend, backend, multiple Laravel workers, MySQL, MinIO, two Python AI services, Prometheus, and Grafana. Initial full build can exceed a short command timeout on a Windows laptop. Once images are built, targeted service startup and HTTP checks are practical for a graduation demo.

## 6.15 Evaluation Limitations

- The AI CV Analyzer pytest suite was not executed because pytest was absent in that container.
- The browser CV upload remains a smoke test, and the mini dataset is synthetic rather than statistically representative.
- The recommendation score shown in screenshots is an estimated local demo output.
- External scraping reliability depends on source availability and changing website/API behavior.
- Production security, privacy, and performance audits remain future work.

## 6.16 Summary of Results

| Area               | Command or Scenario                                                            | Result | Evidence          |
|--------------------|--------------------------------------------------------------------------------|--------|-------------------|
| Docker config      | docker compose config --quiet                                                  | Passed | Terminal evidence |
| Docker prod config | docker compose -f docker-compose.yml -f docker-compose.prod.yml config --quiet | Passed | Terminal evidence |

| Area                  | Command or Scenario                              | Result                           | Evidence       |
|-----------------------|--------------------------------------------------|----------------------------------|----------------|
| Backend dependencies  | composer install --no-interaction -prefer-dist   | Passed                           | Command output |
| Backend routes        | php artisan route:list                           | Passed, 131 routes               | Command output |
| Backend tests         | php artisan test                                 | Passed, 39 tests, 297 assertions | Command output |
| Frontend lint         | ESLint                                           | Passed, 9 warnings, 0 errors     | Command output |
| Frontend build        | Vite build                                       | Passed, 2904 modules transformed | Command output |
| AI Job Miner tests    | python -m pytest                                 | Passed, 75 tests                 | Command output |
| AI CV Analyzer syntax | python -m compileall .                           | Passed                           | Command output |
| AI CV Analyzer pytest | python -m pytest                                 | Skipped, pytest missing          | Command output |
| HTTP probes           | /, /api/health, /api/ready, /status, AI services | 200 responses                    | Command output |

*Table 11. Automated validation results.*

| Test ID | Module          | Scenario                       | Status         | Evidence                       |
|---------|-----------------|--------------------------------|----------------|--------------------------------|
| M-01    | Authentication  | Register demo user             | Passed         | Register screenshot/API output |
| M-02    | Authentication  | Login student                  | Passed         | Figure 12                      |
| M-03    | CV upload       | Upload valid PDF               | Passed         | Figures 13-15                  |
| M-04    | CV upload       | Invalid file handling          | Not Run Manual | Backend validation tests       |
| M-05    | Recommendations | Open jobs page after CV        | Passed         | Figure 16                      |
| M-06    | Gap analysis    | Analyze selected job           | Passed         | Figure 18                      |
| M-07    | Tracker         | Save job                       | Passed         | Figure 19                      |
| M-08    | Admin           | Login admin and open dashboard | Passed         | Figure 22                      |
| M-09    | Admin sources   | Open diagnostics               | Passed         | Figure 24                      |
| M-10    | Status          | Open system status page        | Passed         | Figure 21                      |

*Table 12. Manual functional evaluation matrix.*

| Test ID | Expected vs Actual Observation                                                                                                                    |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| M-01    | Expected user creation and token return; actual user was created with an accepted Gmail-style address.                                            |
| M-02    | Expected dashboard access after login; actual dashboard loaded.                                                                                   |
| M-03    | Expected CV acceptance and parsing; actual upload parsed successfully.                                                                            |
| M-04    | Expected validation error for invalid file; actual manual browser repetition was not run because backend validation/tests already cover the rule. |
| M-05    | Expected recommendations with estimated matches; actual jobs page loaded.                                                                         |
| M-06    | Expected match and skill breakdown; actual gap page loaded.                                                                                       |

| Test ID | Expected vs Actual Observation                                                 |
|---------|--------------------------------------------------------------------------------|
| M-07    | Expected saved job in tracker; actual application page loaded with saved item. |
| M-08    | Expected admin-only dashboard; actual dashboard visible after admin login.     |
| M-09    | Expected source diagnostics; actual diagnostics page visible.                  |
| M-10    | Expected health UI; actual system status page visible.                         |

*Table 13. Manual functional observations.*

## Chapter 7: Security and Privacy

### 7.1 Introduction

Security in CareerCompass is implemented for a graduation/demo context. The system includes meaningful controls, but it should not be presented as production-grade without future hardening, legal review, and operational security work.

### 7.2 Authentication and Authorization

User authentication uses token-based API access through Laravel. Laravel Sanctum supports API token and SPA authentication use cases [2]. CareerCompass stores an auth token in the browser and attaches it to API requests. Authorization is role-aware: student routes require authentication, while admin routes require the admin role. Authentication security should follow established password and session guidance in production [28].

### 7.3 Admin Access Control

Admin access is enforced server-side by admin middleware. Frontend route protection improves user experience, but the backend check is the important control. The demo admin account is generated by a seeder and should be changed or disabled in any non-demo environment.

### 7.4 CV File Privacy

CV files contain personal data. CareerCompass validates file type and size, stores files privately, and avoids public direct file exposure. OWASP's file upload guidance emphasizes validation, restricted storage, and safe handling [27].

### 7.5 Private Storage and Signed Downloads

Uploaded CV files are stored through private storage and accessed through signed or temporary URLs. MinIO/S3-compatible storage supports object-based file storage and access control patterns [9]. The graduation demo should still avoid uploading real sensitive CVs unless the environment is controlled.

### 7.6 Internal Service Tokens

Scraper import routes use a service token middleware. This reduces accidental public ingestion, but service tokens must be rotated, stored securely, and monitored in production.

### 7.7 Payload and File Validation

Laravel form requests validate registration, login, CV upload, applications, and profile updates. File validation includes MIME type, extension, and size checks. Validation reduces risk but does not replace malware scanning, deep file inspection, or content disarm in production.

### 7.8 Logging and Request IDs

The API client attaches request IDs, and backend logging records important events such as CV processing status and AI gateway errors. For production, logs should avoid sensitive CV content and should be retained according to a privacy policy.

### 7.9 Demo Security Limitations

| Area          | Current Demo Control | Production Hardening Needed          |
|---------------|----------------------|--------------------------------------|
| Admin account | Demo seeder account  | Secret rotation, SSO/MFA, audit logs |

| Area            | Current Demo Control            | Production Hardening Needed                               |
|-----------------|---------------------------------|-----------------------------------------------------------|
| CV files        | Private storage and signed URLs | Malware scanning, retention policy, consent model         |
| Tokens          | Bearer tokens                   | Token rotation, secure cookie strategy, revocation review |
| Scraper service | Internal token                  | Secret manager, network isolation, rate limits            |
| Monitoring      | Local Prometheus/Grafana        | Auth, TLS, dashboard access control                       |
| Privacy         | Local demo posture              | Legal review, privacy notice, data minimization           |

*Table 14. Security and privacy controls.*

## 7.10 Future Production Hardening

Future work should include HTTPS-only deployment, secure cookie/session strategy, administrator MFA, centralized secrets management, object scanning, retention policies, audit logging, rate-limit review, CSRF/CORS review, dependency vulnerability scanning, and a privacy impact assessment.

## Chapter 8: Conclusion and Future Work

### 8.1 Conclusion

CareerCompass demonstrates a practical AI-assisted career guidance workflow for a graduation project. It integrates CV parsing, profile normalization, skill extraction, imported job records, estimated recommendation, gap analysis, application tracking, admin diagnostics, and containerized deployment.

### 8.2 Project Achievements

- Built a working multi-service web application with frontend, backend, AI services, database, object storage, proxy, and monitoring.
- Implemented student authentication, dashboard, CV upload, profile view, jobs, gap analysis, and application tracking.
- Implemented admin dashboard, job management, source diagnostics, and target role management.
- Added tests and validation commands across backend, frontend, Python services, Docker, and HTTP probes.
- Captured browser screenshots from the running local system.
- Generated a formal report with diagrams, references, and evaluation notes.

### 8.3 Educational Value

The project demonstrates practical learning in software architecture, service decomposition, secure file handling, API design, database schema design, frontend state management, AI service integration, testing, Docker operations, monitoring, and academic documentation.

### 8.4 Current Limitations

- The system is a graduation/demo platform and not a production product.
- Recommendation and gap analysis outputs are estimates.
- AI evaluation needs larger labeled datasets and repeatable scoring.
- External scraping sources can be unstable.
- The AI CV Analyzer container needs pytest installed to run its test suite.
- Security and privacy controls need production hardening before real deployment.

### 8.5 Future Work

- Add a larger CV/job evaluation dataset with manual labels.
- Improve role taxonomy and skill normalization.
- Add production-grade authentication and administrator controls.
- Add malware scanning and retention policies for uploaded CV files.
- Improve recommendation explanations and calibration.
- Add automated browser end-to-end tests.
- Extend CI to run all containerized test suites consistently.
- Improve observability dashboards and alerting.
- Add deployment documentation for a secure cloud environment.

### 8.6 Final Remarks

CareerCompass is an original graduation project that connects academic software engineering requirements with a practical career guidance problem. Its strongest value is the integration of many

realistic system parts into one demonstrable workflow, while preserving honest language about limitations and future production work.

## References

- [1] Laravel, "Laravel 12.x Documentation," Laravel, 2026. [Online]. Available: <https://laravel.com/docs/12.x>. Accessed: May 29, 2026.
- [2] Laravel, "Laravel Sanctum," Laravel, 2026. [Online]. Available: <https://laravel.com/docs/12.x/sanctum>. Accessed: May 29, 2026.
- [3] Laravel, "Queues," Laravel, 2026. [Online]. Available: <https://laravel.com/docs/12.x/queues>. Accessed: May 29, 2026.
- [4] Meta Open Source, "Quick Start - React," React Documentation, 2026. [Online]. Available: <https://react.dev/learn>. Accessed: May 29, 2026.
- [5] Vite, "Getting Started," Vite Documentation, 2026. [Online]. Available: <https://vite.dev/guide/>. Accessed: May 29, 2026.
- [6] FastAPI, "FastAPI Documentation," FastAPI, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>. Accessed: May 29, 2026.
- [7] Python Software Foundation, "Python 3 Documentation," Python, 2026. [Online]. Available: <https://docs.python.org/3/>. Accessed: May 29, 2026.
- [8] Oracle, "MySQL 8.0 Reference Manual," MySQL Documentation, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. Accessed: May 29, 2026.
- [9] MinIO, "MinIO AIStor Documentation," MinIO Documentation, 2026. [Online]. Available: <https://docs.min.io/aistor/>. Accessed: May 29, 2026.
- [10] Docker, "What is a Container?," Docker Resources, 2026. [Online]. Available: <https://www.docker.com/resources/what-container/>. Accessed: May 29, 2026.
- [11] Docker, "Docker Compose," Docker Documentation, 2026. [Online]. Available: <https://docs.docker.com/compose/>. Accessed: May 29, 2026.
- [12] Nginx, "nginx Documentation," nginx, 2026. [Online]. Available: <https://nginx.org/en/docs/>. Accessed: May 29, 2026.
- [13] Prometheus, "Overview," Prometheus Documentation, 2026. [Online]. Available: <https://prometheus.io/docs/introduction/overview/>. Accessed: May 29, 2026.
- [14] Grafana Labs, "Grafana OSS and Enterprise," Grafana Documentation, 2026. [Online]. Available: <https://grafana.com/docs/grafana/latest/>. Accessed: May 29, 2026.
- [15] GitHub, "GitHub Actions Documentation," GitHub Docs, 2026. [Online]. Available: <https://docs.github.com/en/actions>. Accessed: May 29, 2026.
- [16] MDN Web Docs, "HTTP: Hypertext Transfer Protocol," MDN, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP>. Accessed: May 29, 2026.
- [17] Scrapy, "Scrapy Documentation," Scrapy, 2026. [Online]. Available: <https://docs.scrapy.org/en/latest/>. Accessed: May 29, 2026.
- [18] Leonard Richardson, "Beautiful Soup Documentation," Beautiful Soup, 2026. [Online]. Available: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>. Accessed: May 29, 2026.

- [19] scikit-learn, "TfidfVectorizer," scikit-learn Documentation, 2026. [Online]. Available: [https://scikit-learn.org/stable/modules/generated/sklearn.feature\\_extraction.text.TfidfVectorizer.html](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html). Accessed: May 29, 2026.
- [20] scikit-learn, "cosine\_similarity," scikit-learn Documentation, 2026. [Online]. Available: [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine\\_similarity.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html). Accessed: May 29, 2026.
- [21] UKP Lab, "Sentence Transformers Documentation," Sentence Transformers, 2026. [Online]. Available: <https://www.sbert.net/>. Accessed: May 29, 2026.
- [22] Artifex, "PyMuPDF Documentation," PyMuPDF, 2026. [Online]. Available: <https://pymupdf.readthedocs.io/en/latest/>. Accessed: May 29, 2026.
- [23] jsvine, "pdfplumber," GitHub Repository, 2026. [Online]. Available: <https://github.com/jsvine/pdfplumber>. Accessed: May 29, 2026.
- [24] Jaided AI, "EasyOCR," GitHub Repository, 2026. [Online]. Available: <https://github.com/JaidedAI/EasyOCR>. Accessed: May 29, 2026.
- [25] PHPUnit, "PHPUnit 12.0 Manual," PHPUnit Documentation, 2026. [Online]. Available: <https://docs.phpunit.de/en/12.0/>. Accessed: May 29, 2026.
- [26] pytest, "pytest Documentation," pytest, 2026. [Online]. Available: <https://docs.pytest.org/en/stable/>. Accessed: May 29, 2026.
- [27] OWASP, "File Upload Cheat Sheet," OWASP Cheat Sheet Series, 2026. [Online]. Available: [https://cheatsheetseries.owasp.org/cheatsheets/File\\_Upload\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html). Accessed: May 29, 2026.
- [28] OWASP, "Authentication Cheat Sheet," OWASP Cheat Sheet Series, 2026. [Online]. Available: [https://cheatsheetseries.owasp.org/cheatsheets/Authentication\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html). Accessed: May 29, 2026.
- [29] Martin Fowler and James Lewis, "Microservices," martinowler.com, 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>. Accessed: May 29, 2026.
- [30] scikit-learn, "precision\_recall\_fscore\_support," scikit-learn Documentation, 2026. [Online]. Available: [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision\\_recall\\_fscore\\_support.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_fscore_support.html). Accessed: May 29, 2026.

# Appendices

## Appendix A: API Endpoint Summary

| Group            | Example Endpoints                                                                                             | Purpose                                           |
|------------------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| Health           | /api/health, /api/ready, /api/metrics                                                                         | Liveness, readiness, and Prometheus metrics.      |
| Auth             | /api/v1/register, /api/v1/login, /api/v1/logout, /api/v1/user                                                 | User identity and token lifecycle.                |
| CV               | /api/v1/upload-cv, /api/v1/user/skills, /api/v1/user/cv-analysis                                              | CV upload, parsed analysis, and extracted skills. |
| Jobs             | /api/v1/jobs, /api/v1/jobs/recommended, /api/v1/jobs/{id}                                                     | Job listing, details, and recommendations.        |
| Gap Analysis     | /api/v1/gap-analysis/job/{jobId}, /api/v1/gap-analysis/role/{roleId}                                          | Skill comparison and recommendations.             |
| Applications     | /api/v1/applications                                                                                          | Save and update tracked opportunities.            |
| Admin            | /api/v1/admin/dashboard/stats, /api/v1/admin/jobs, /api/v1/admin/scraping-sources, /api/v1/admin/target-roles | Admin dashboards and diagnostics.                 |
| Internal Scraper | /api/jobs/import, /api/jobs/import/check, /api/proxies/active                                                 | Service-token protected import routes.            |

Table 15. API endpoint summary.

## Appendix B: Database Tables Summary

| Table            | Purpose                                                                                      |
|------------------|----------------------------------------------------------------------------------------------|
| users            | Authentication identity, role, and banned status.                                            |
| profiles         | Student profile details and contact metadata.                                                |
| skills           | Canonical skill catalog.                                                                     |
| user_skill       | User-to-skill relationship and proficiency metadata.                                         |
| experiences      | Extracted or entered experience records.                                                     |
| cv_analyses      | CV parsing status, predicted role, confidence, file metadata, strengths, gaps, and warnings. |
| job_postings     | Imported/demo job data.                                                                      |
| applications     | Saved opportunities and status tracking.                                                     |
| scraping_sources | Admin-configured job source definitions.                                                     |
| target_job_roles | Target role list for scraping and market exploration.                                        |
| scraping_jobs    | Scraping batch execution state.                                                              |

Table 16. Database tables summary.

## Appendix C: Docker Services Summary

| Service                 | Role                                                               |
|-------------------------|--------------------------------------------------------------------|
| nginx                   | Public gateway and reverse proxy.                                  |
| frontend                | React/Vite web UI container.                                       |
| backend-api             | Laravel API runtime.                                               |
| backend-worker variants | Queue processing for default, high, AI, emails, and scraping work. |
| backend-scheduler       | Scheduled Laravel commands.                                        |
| db                      | MySQL database.                                                    |
| minio                   | S3-compatible CV object storage.                                   |
| ai-cv-analyzer          | FastAPI CV parsing service.                                        |
| ai-job-miner            | FastAPI job mining service.                                        |
| prometheus              | Metrics collection.                                                |
| grafana                 | Metrics visualization.                                             |

Table 17. Docker services summary.

## Appendix D: Screenshots

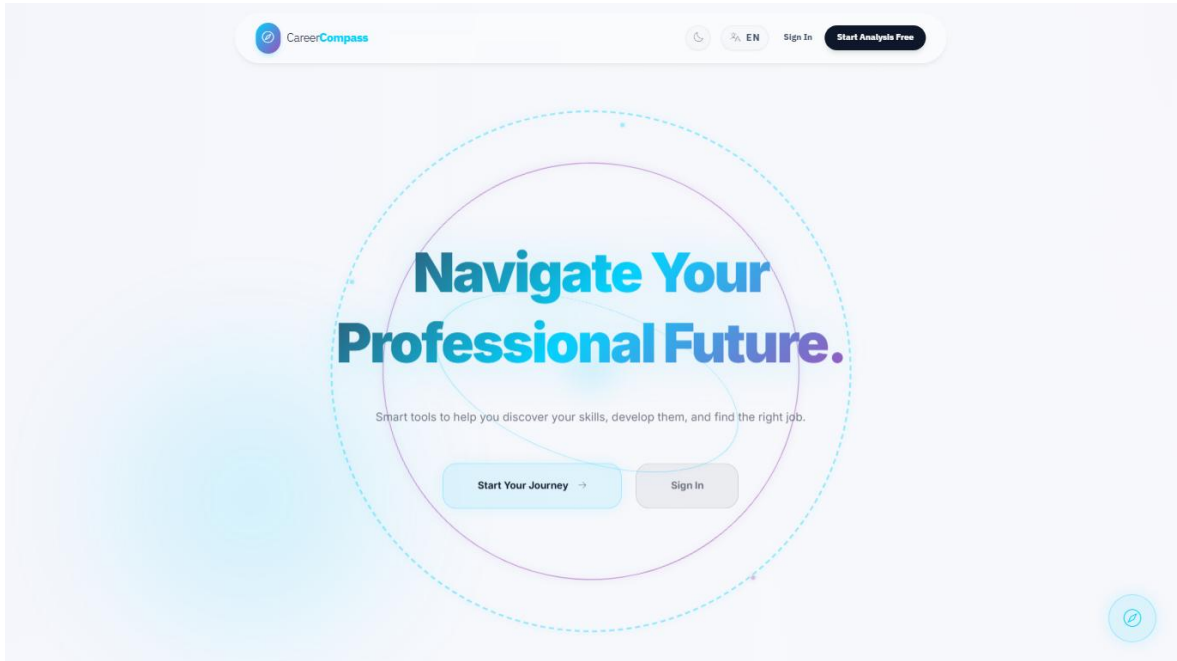


Figure 9. Home page.

 CareerCompass
 


 EN
 [Sign In](#)
[Start Analysis Free](#)



## Join CareerCompass

Create your account and start your professional journey.

STEP 1 OF 2

IDENTITY

FULL NAME

EMAIL ADDRESS

[Next Step →](#)

Already have an account? [Sign In](#)

Figure 10. Register page.

 CareerCompass
 


 EN
 [Sign In](#)
[Start Analysis Free](#)

 CareerCompass
 

## Welcome back to your professional journey.

Log in to access your CV-based career dashboard, continue your skills analysis, and explore imported opportunities.

 [Graduation demo workspace](#)

## Sign In

Please enter your credentials to proceed.

EMAIL ADDRESS

PASSWORD [Forgot Password?](#)

[Sign In →](#)

Don't have an account? [Create an account](#)

 SECURE CONNECTION

Figure 11. Login page.

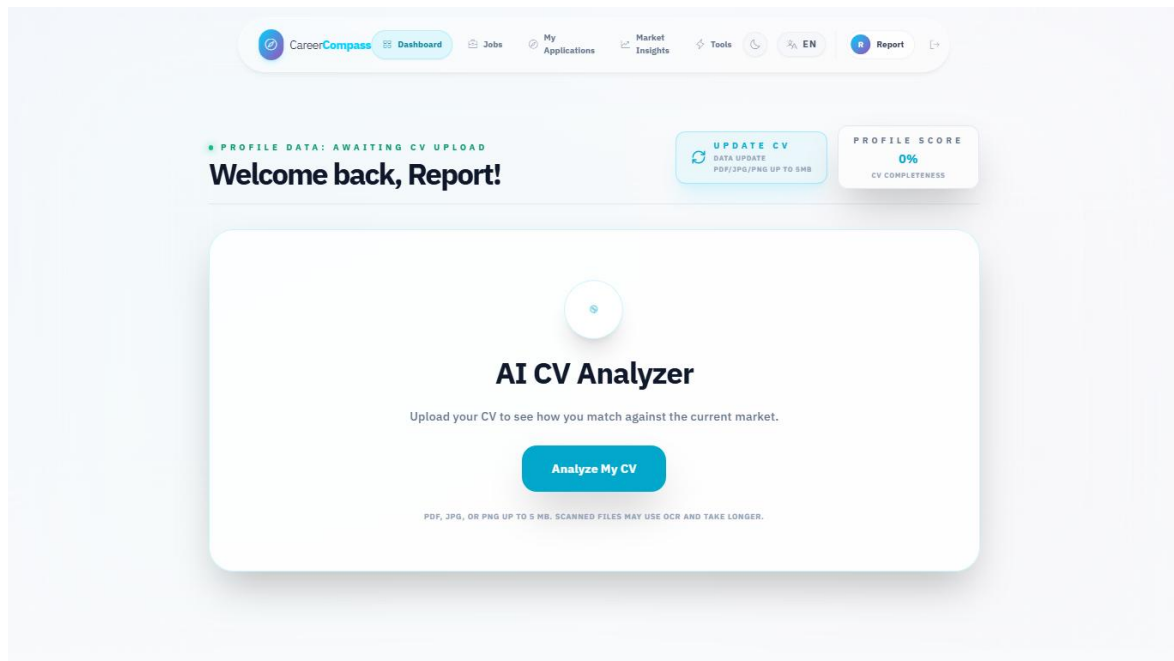


Figure 12. Student dashboard before CV upload.

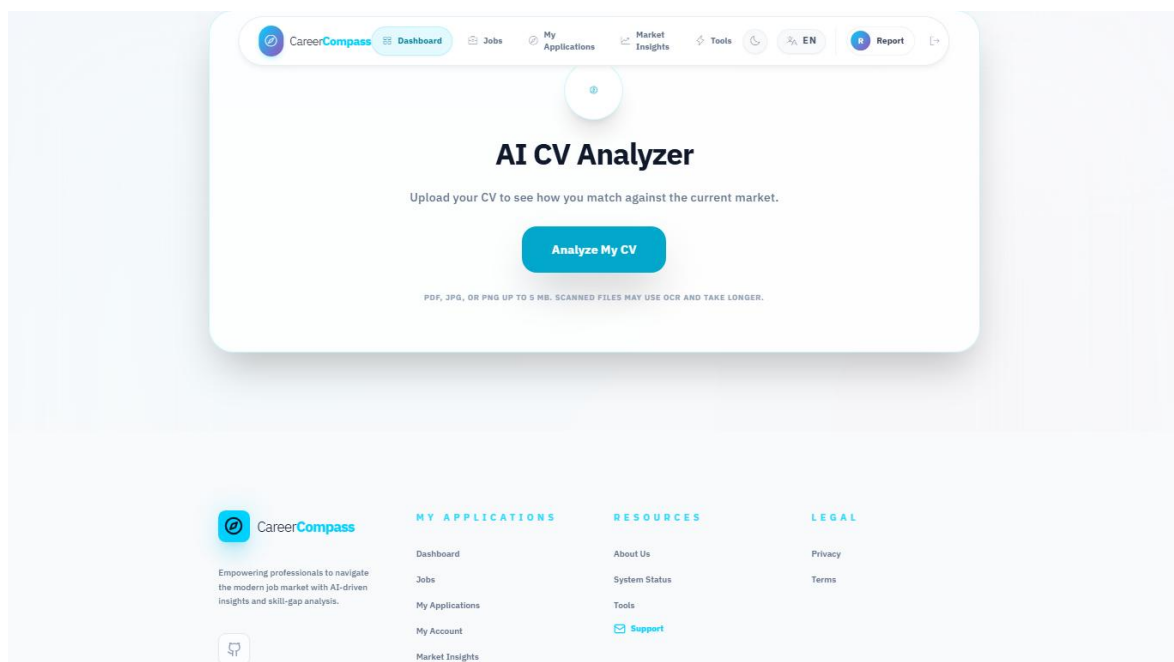


Figure 13. CV upload user interface.

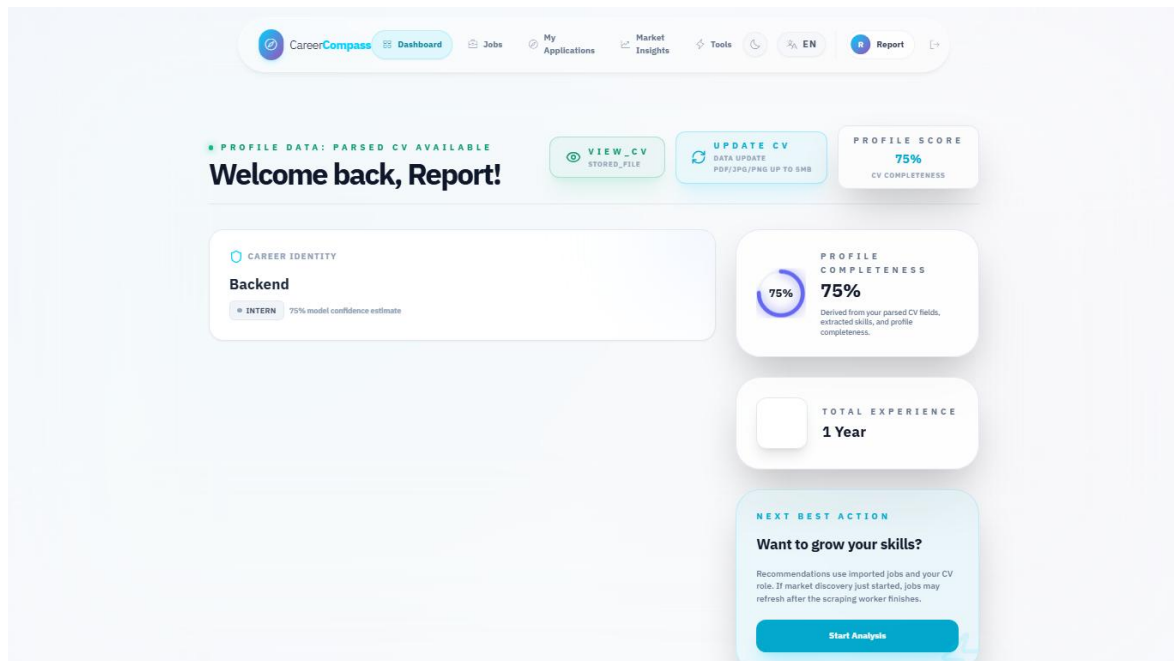


Figure 14. Dashboard after successful CV parsing.

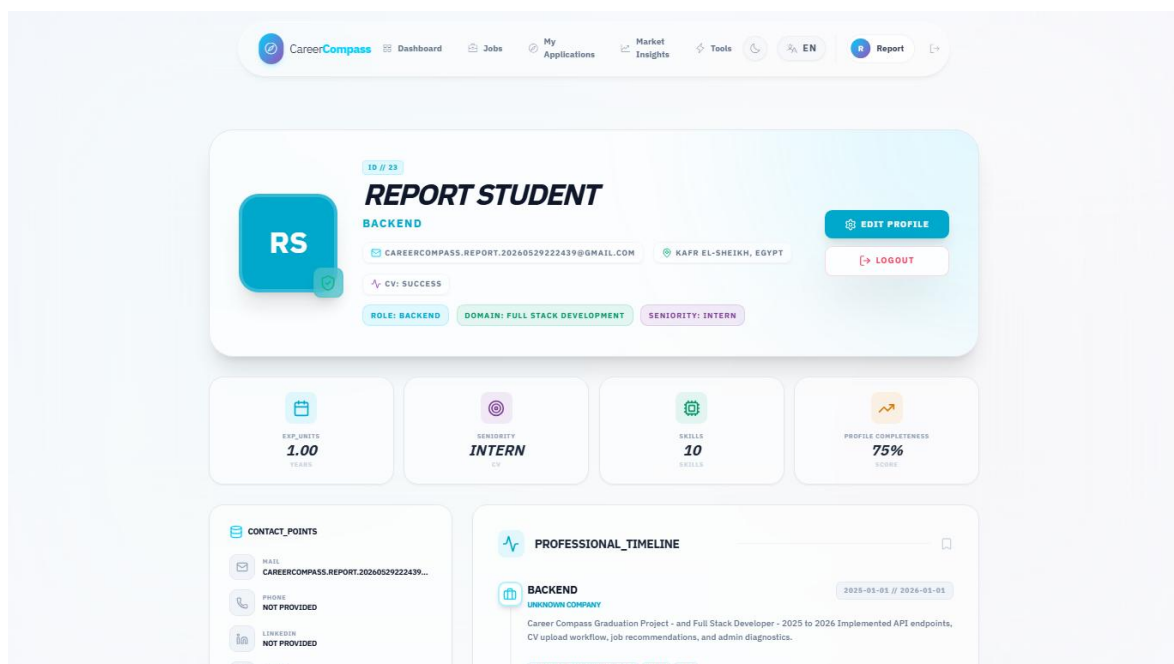


Figure 15. Extracted profile and skills page.

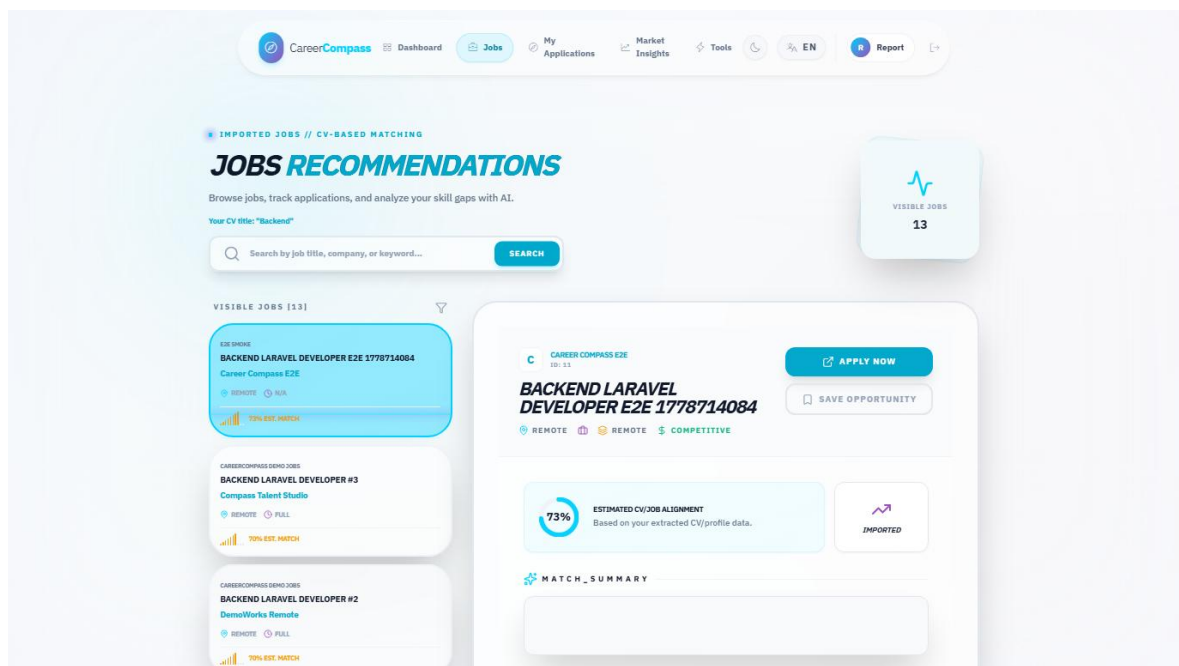


Figure 16. Jobs recommendations page.

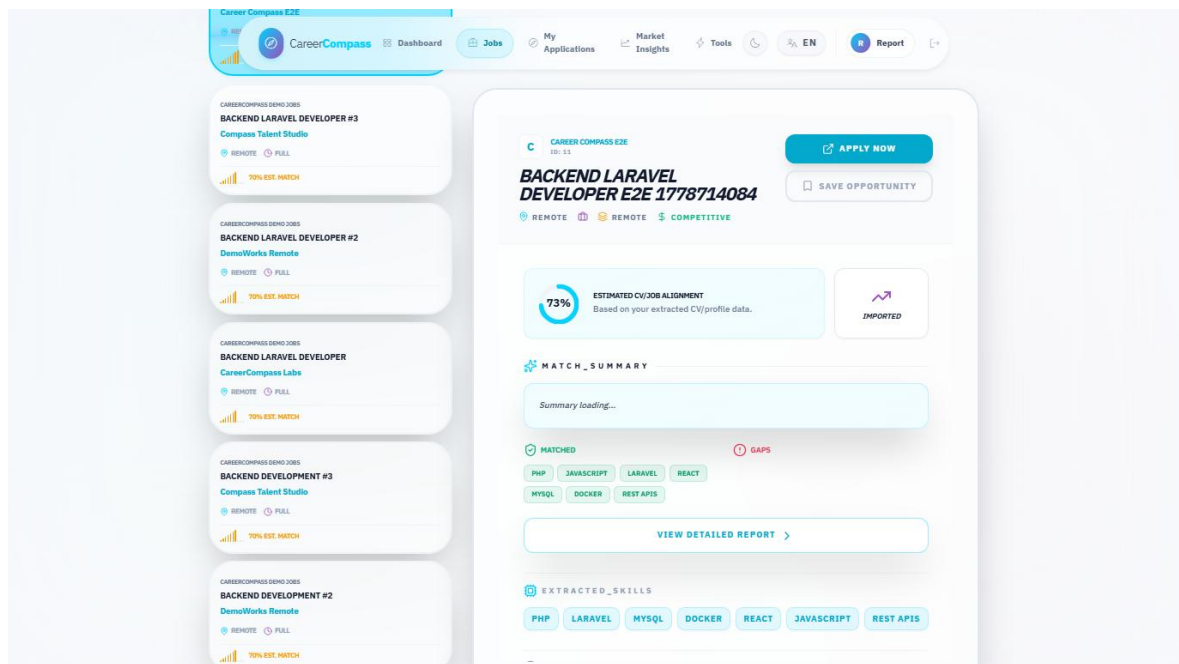


Figure 17. Job detail and inline gap panel.

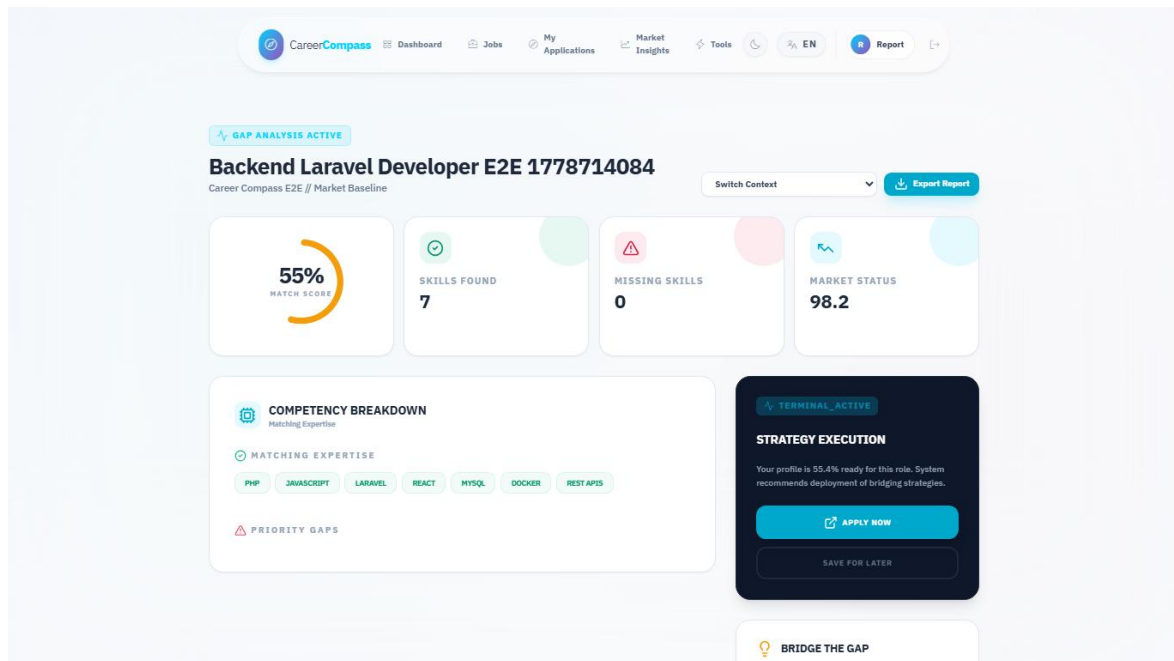


Figure 18. Gap analysis page.

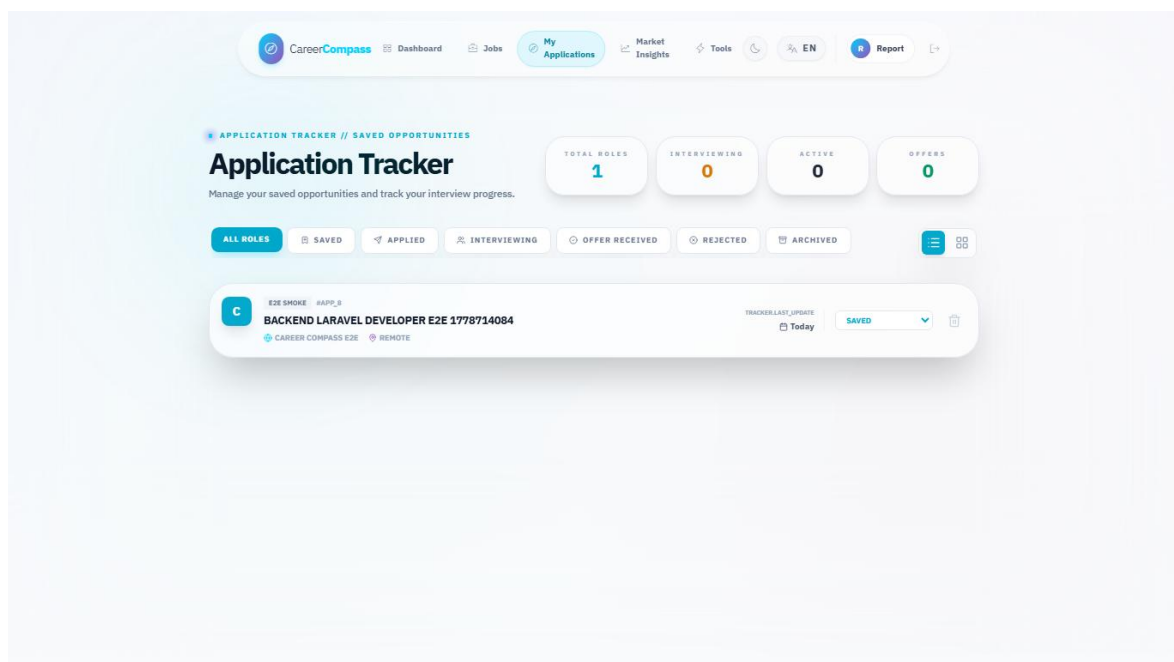


Figure 19. Applications tracker page.

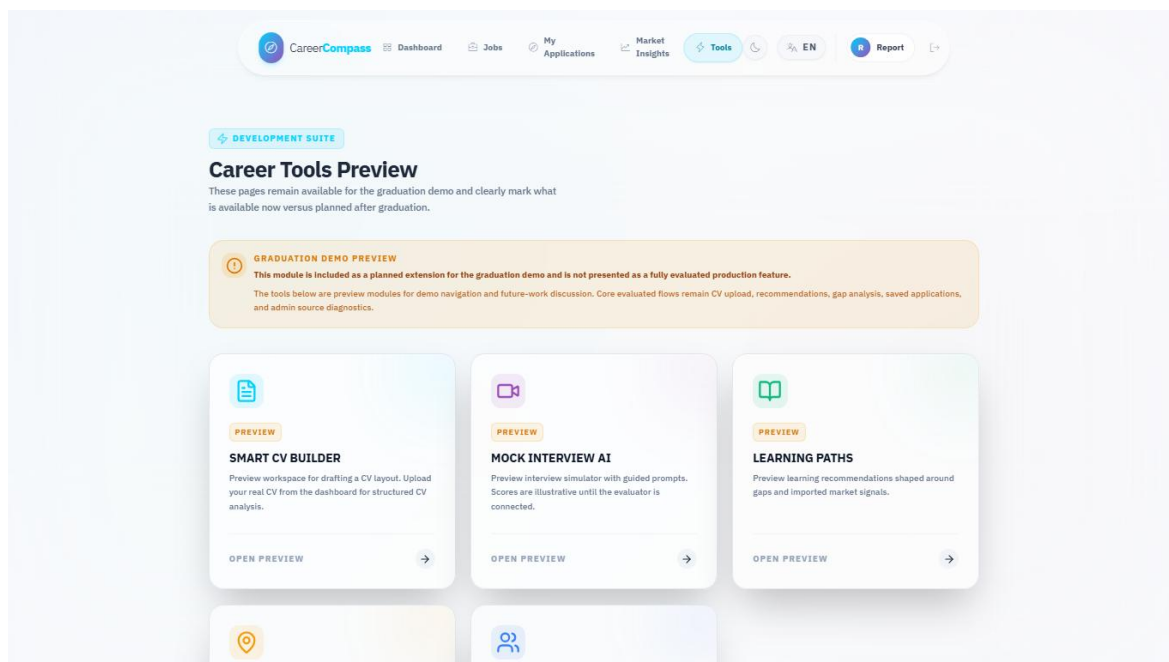


Figure 20. Tools Hub preview page.

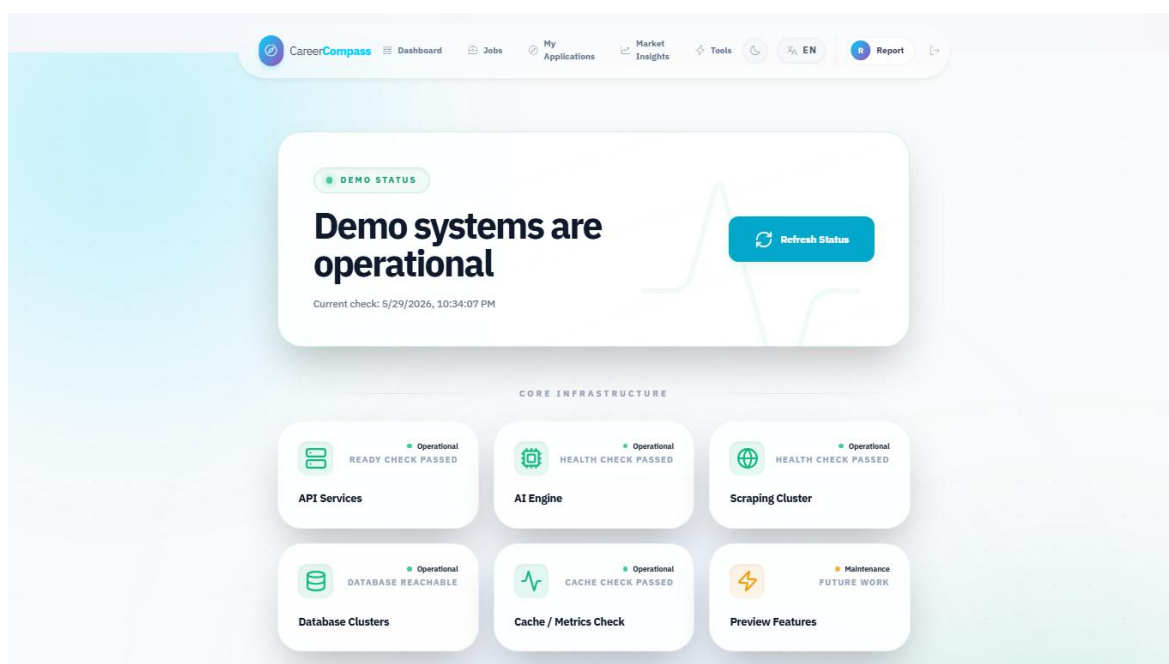


Figure 21. System status page.

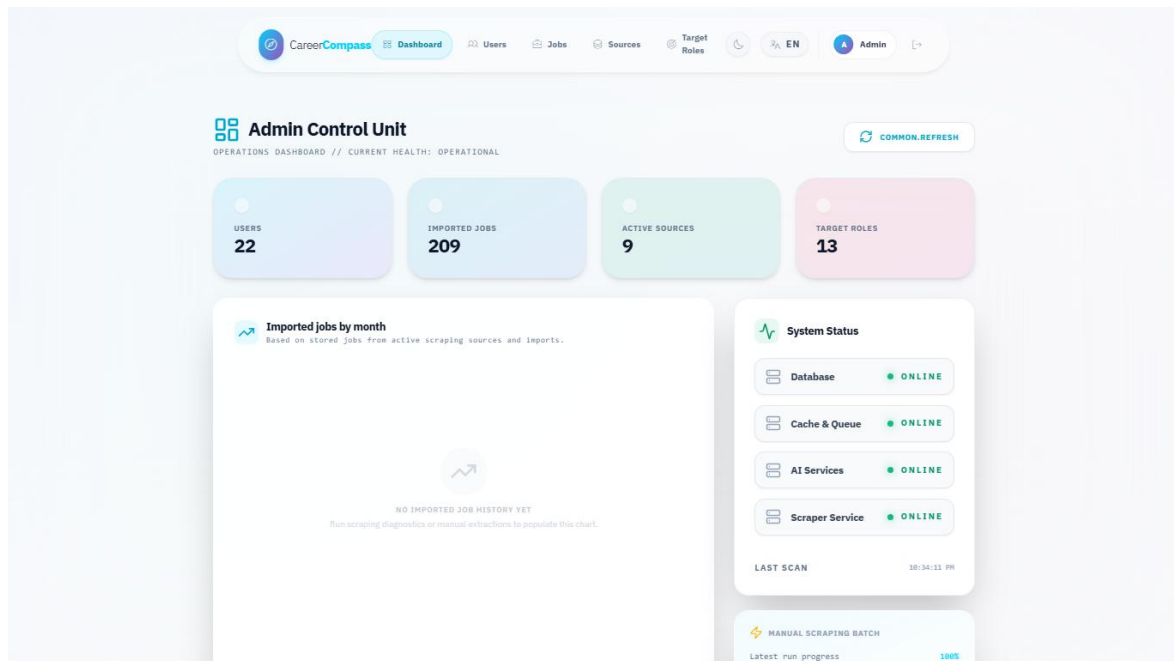


Figure 22. Admin dashboard.

**Imported Jobs**  
// IMPORTED\_JOBS\_INDEX

Search mentors by name, role, or co **DLQ MONITOR**

| IMPORTED JOBS                                                                                                              | JOBS.COMPANY                                | JOBS.LOCATION                 | STATUS  | OPERATIONS |
|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|-------------------------------|---------|------------|
| <b>Registered Nurse (Rn) - Hiring Now!</b><br>Adzuna US Tech • 233...                                                      | Research Medical Center                     | Mission Hills, Johnson County | INDEXED | → 🗑️       |
| <b>Dentist (Oral Surgery)</b><br>Adzuna US Tech • 232...                                                                   | Spencer Dental & Braces - a Benevis company | Bornack, Roanoke              | INDEXED | → 🗑️       |
| <b>Systemadministrator (M/W/D)</b><br>Arbeitsnow Job Board • 231...                                                        | MY Humancapital GmbH                        | Munich                        | INDEXED | → 🗑️       |
| <b>(Junior) HR Manager (M/W/D) - Schwerpunkt Recruiting</b><br>Arbeitsnow Job Board • 230...                               | MY Humancapital GmbH                        | Munich                        | INDEXED | → 🗑️       |
| <b>Account Manager Influencer Marketing Tiktok, Youtube &amp; Instagram (All Genders)</b><br>Arbeitsnow Job Board • 229... | Ykone GmbH                                  | Munich                        | INDEXED | → 🗑️       |
| <b>Junior Research Consultant (M/W/D)</b><br>Arbeitsnow Job Board • 228...                                                 | mindline GmbH                               | Hamburg                       | INDEXED | → 🗑️       |
| <b>Praktikum Online-Marketing / Marketing / Ki / AI</b><br>Arbeitsnow Job Board • 227...                                   | SEOMATIK GmbH                               | Stiefeld                      | INDEXED | → 🗑️       |
| <b>Praktikum Online-Marketing / Marketing / Startup</b><br>Arbeitsnow Job Board • 226...                                   | SEOMATIK GmbH                               | Stiefeld                      | INDEXED | → 🗑️       |

Figure 23. Admin jobs page.

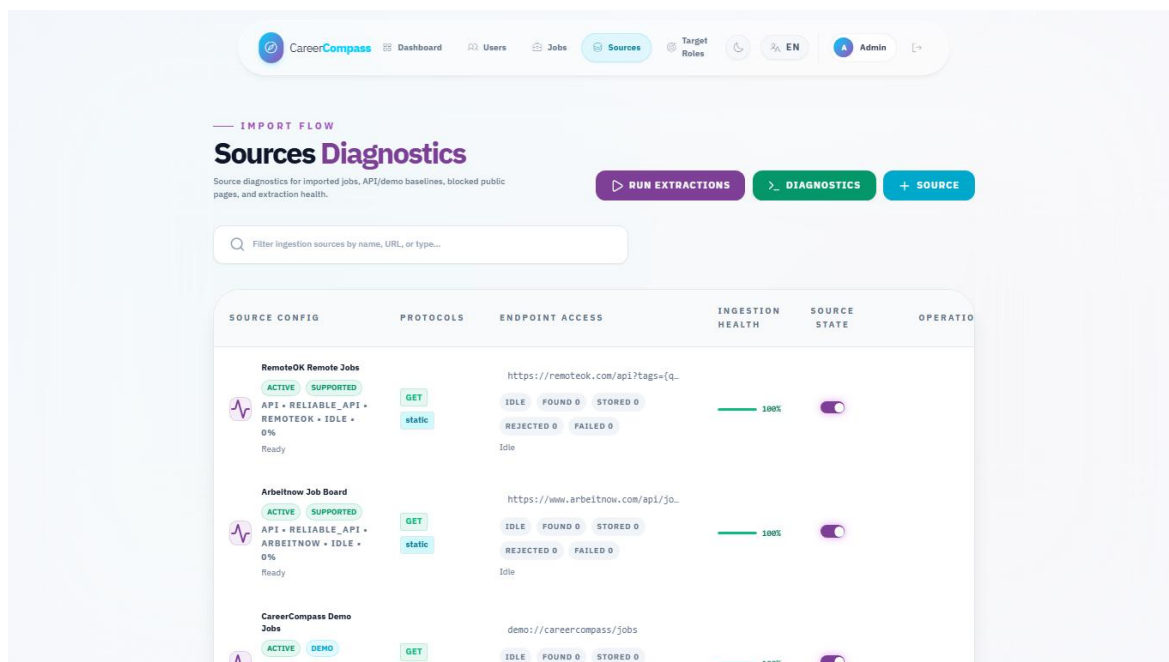


Figure 24. Admin sources diagnostics page.

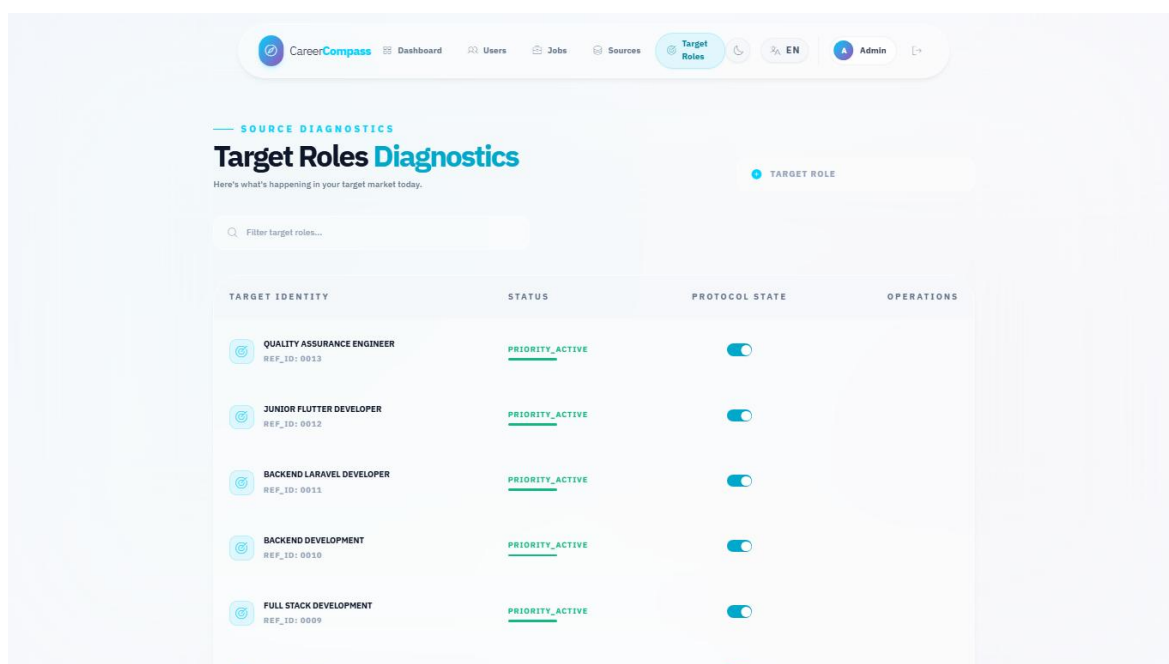


Figure 25. Admin target roles page.

| Docker Compose Services Evidence |                                            |                             |                         |              |
|----------------------------------|--------------------------------------------|-----------------------------|-------------------------|--------------|
| NAME                             | IMAGE                                      | COMMAND                     | SERVICE                 | CREATED      |
| cc-backend                       | graduation-project-backend-api             | "/usr/local/bin/cc-eâ€¦!"   | backend-api             | 18 hours ago |
| cc-backend-scheduler             | graduation-project-backend-scheduler       | "/usr/local/bin/cc-eâ€¦!"   | backend-scheduler       | 18 hours ago |
| cc-backend-worker                | graduation-project-backend-worker          | "/usr/local/bin/cc-eâ€¦!"   | backend-worker          | 18 hours ago |
| cc-backend-worker-ai             | graduation-project-backend-worker-ai       | "/usr/local/bin/cc-eâ€¦!"   | backend-worker-ai       | 18 hours ago |
| cc-backend-worker-emails         | graduation-project-backend-worker-emails   | "/usr/local/bin/cc-eâ€¦!"   | backend-worker-emails   | 18 hours ago |
| cc-backend-worker-high           | graduation-project-backend-worker-high     | "/usr/local/bin/cc-eâ€¦!"   | backend-worker-high     | 18 hours ago |
| cc-backend-worker-scraping       | graduation-project-backend-worker-scraping | "/usr/local/bin/cc-eâ€¦!"   | backend-worker-scraping | 18 hours ago |
| cc-cv-analyzer                   | graduation-project-ai-cv-analyzer          | "uvicorn main:app --â€¦!"   | ai-cv-analyzer          | 18 hours ago |
| cc-db                            | mysql:8.0                                  | "docker-entrypoint.sâ€¦!"   | db                      | 2 weeks ago  |
| cc-frontend                      | graduation-project-frontend                | "/docker-entrypoint.â€¦!"   | frontend                | 18 hours ago |
| cc-job-miner                     | graduation-project-ai-job-miner            | "uvicorn service_apiâ€¦!"   | ai-job-miner            | 18 hours ago |
| cc-nginx                         | nginx:stable-alpine                        | "/docker-entrypoint.â€¦!"   | nginx                   | 6 days ago   |
| graduation-project-grafana-1     | grafana/grafana-oss:11.4.0                 | "/run.sh"                   | grafana                 | 2 weeks ago  |
| graduation-project-minio-1       | minio/minio:latest                         | "/usr/bin/docker-entâ€¦!"   | minio                   | 2 weeks ago  |
| graduation-project-prometheus-1  | prom/prometheus:v2.55.1                    | "/bin/sh -c 'sed \"s/â€¦!'" | prometheus              | 2 weeks ago  |

Figure 26. Docker services evidence.

| Validation Command Evidence                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| Validation summary captured for the graduation book:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |  |
| <pre> docker compose config --quiet: PASSED docker compose -f docker-compose.yml -f docker-compose.prod.yml config --quiet: PASSED docker compose up -d --build: stack built; full initial build exceeded 15 minutes, then targeted rebuild/start completed composer install in backend-api container: PASSED php artisan config:clear: PASSED php artisan route:list: PASSED (131 routes) php artisan migrate --force --no-interaction: PASSED (nothing to migrate) php artisan test: PASSED (39 tests, 297 assertions) frontend ESLint: PASSED with 9 warnings and 0 errors frontend Vite build: PASSED (2904 modules transformed) ai-job-miner pytest: PASSED (75 passed) ai-cv-analyzer compileall: PASSED ai-job-miner compileall: PASSED ai-cv-analyzer pytest: SKIPPED/blocked because pytest is not installed in that container HTTP probes: /, /api/health, /api/ready, /status, AI CV Analyzer root, and Job Miner health returned 200 </pre> |  |

Figure 27. Validation command evidence.

## Appendix E: Test Cases

The manual test matrix in Chapter 6 should be repeated before final submission. Additional recommended tests include invalid CV uploads, banned user login, expired signed download URLs, failed AI service behavior, scraper token rejection, admin route rejection for normal users, and browser checks on a clean database.

The mini dataset evaluation files are:

- evaluation/mini\_cv\_dataset.json
- evaluation/mini\_jobs\_dataset.json
- evaluation/expected\_labels.json
- evaluation/run\_mini\_evaluation.py
- evaluation/mini\_evaluation\_results.json
- evaluation/mini\_evaluation\_summary.md

## Appendix F: GitHub Actions / CI Summary

The repository contains GitHub Actions workflow definitions. After the draft PR is opened, reviewers should inspect PR checks, rerun failed jobs if needed, and confirm that branch protection expectations are satisfied before merging. A CI screenshot was not embedded in this generated report because live PR checks were not available until after branch push.

## Appendix G: Demo Script

1. Start Docker Desktop.
2. Run `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`.
3. Open `http://localhost`.
4. Register or login as a student.
5. Upload a sample PDF CV from the dashboard.
6. Review parsed profile and skills.
7. Open Jobs and select a recommendation.
8. Open Gap Analysis and explain matched/missing skills.
9. Save the job to Applications.
10. Login as the demo admin.
11. Open Admin Dashboard, Jobs, Sources, and Target Roles.
12. Open System Status and explain health checks.
13. Discuss limitations and future work honestly.